

АНАЛИЗ ПАТЕНТА US 7502723B1

Asymmetric Minor Hysteresis Loop Model and Circuit Simulator Including the Same

Физический смысл, полная логика алгоритма, анализ рисунков
и описание реализации на Python

Дмитрий Махновский

Исследовательский Центр ФИНГО, Москва

11 сентября 2026 г.

Содержание

1. Назначение отчёта и исходные материалы	4
2. Что именно решает патент Engelhardt.....	4
2.1. Контекст: модели гистерезиса Jiles-Atherton и Chan	4
2.2. Идея улучшения модели гистерезиса Chan	4
3. Математическая основа модели.....	5
3.1. Материальные параметры магнитного сердечника	5
3.2. Верхняя и нижняя ветви мажорной петли, и начальная кривая намагничивания	5
3.3. Обычные симметричные минорные ветви и параметр B_d	6
3.4. Базисные функции для аффинного исправления	6
3.5. Аффинное преобразование.....	6
3.6. Ограничивающие функции и смысл B_{d_turn}	7
3.7. Точка H_0	7
3.8. Формулы scale и offset, использованные в Python.....	7
4. Анализ рисунков патента.....	8
4.1. FIG. 1 — дефект модели Chan	8
4.2. FIG. 2A — геометрия нормального и проблемного разворота	8
4.3. FIG. 2B — результат нового метода.....	10
4.4. FIG. 3A — ветвь алгоритма при возрастающем H	11
4.5. FIG. 3B — ветвь алгоритма при уменьшающемся H	12
4.6. FIG. 4 — пример симуляции схемы	19
4.7. FIG. 5A и FIG. 5B — влияние модели на ток индуктора	19
4.8. FIG. 6 — вычислительная реализация	20
5. Полный алгоритм FIG. 3: шаги 300-372	21
5.1. Общие шаги 300-306.....	21
5.2. Возрастающее H : шаги 308-316 — обработка разворота.....	21
5.3. Шаг 312 — построение асимметричной ветви для $H \uparrow$	21
5.4. Шаги 318-332 — выбор формулы при $H \uparrow$	21
5.5. Уменьшающееся H : шаги 348-356 — зеркальный разворот	22
5.6. Шаги 358-372 — выбор формулы при $H \downarrow$	22
6. Формулы изобретения (patent claims)	23
7. Python-проект для расчета петель гистерезиса.....	23
7.1 Структура проекта.....	23
7.2 Модуль функций $BH.py$	24
7.3 Управляющий цикл $main.py$	26
7.4 Тестовые функции из модуля $Hscan.py$	27
7.5 Загрузка параметров в $Parameters.py$	27
7.6 Соответствие патента и Python-реализации.....	28

7.7 Проверки, тесты и диагностические инварианты.....	28
7.8 Замечания к текущей реализации и рекомендации	29
7.9 Результаты расчета.....	30
7.10 Примеры численных расчетов петель гистерезиса	30
8. Python-проект для расчета переходных токов в индукторе с магнитным сердечником	32
8.1 Структура Python-проекта	34
8.2 Начальное состояние	34
8.3 Пробное вычисление магнитного состояния.....	35
8.4 Сохранение состояния гистерезиса при поиске корня	36
8.5 Функция невязки.....	36
8.6 Поиск интервала для бисекции.....	36
8.7 Метод бисекции	37
8.8 Фиксация нового состояния.....	38
8.9 Источник напряжения и параметры	38
8.10 Результаты расчёта.....	39
8.11 Примеры расчетов переходного тока в индукторе и сравнение с LTspice	39
Источники.....	43

1. Назначение отчёта и исходные материалы

Настоящий отчёт систематизирует модель асимметричных минорных петель гистерезиса, предложенную [Michael Thomas Engelhardt](#) в патенте [US 7502723B1](#), [1] и сопоставляет её с текущей Python-реализацией проекта. Основная цель — получить документ, по которому можно независимо восстановить логику модели, проверить соответствие программы блок-схеме патента и продолжить разработку без повторного чтения патента «с нуля».

Патент выдан 10 марта 2009 г.; заявка подана 1 сентября 2005 г. Правообладателем указана [Linear Technology Corporation](#). Предмет патента — вычислительно лёгкая модель ферромагнитного сердечника индуктора или трансформатора, способная корректно описывать асимметричные минорные петли и, в частности, устранять нефизическое поведение модели [Chan](#), [2] при котором минорные ветви могут выйти за границы мажорной петли.

Все рисунки патента в отчёте воспроизведены непосредственно из исходного PDF. Они не перерисовывались и не аннотировались. Для рисунков, размещённых в исходном патенте боком, выполнен только поворот при компоновке страницы, чтобы сделать надписи читаемыми; содержание и графика рисунков не изменялись.

2. Что именно решает патент Engelhardt

2.1. Контекст: модели гистерезиса Jiles-Atherton и Chan

В разделе BACKGROUND патент противопоставляет два подхода. Модель [Jiles-Atherton](#) [3] рассматривается как физически содержательная и точная, но вычислительно более тяжёлая: она использует трансцендентные функции, менее удобно параметризуется стандартными паспортными величинами ферромагнетика и, по утверждению патента, труднее применяется к сердечникам с зазором. [Модель Chan](#), [2] напротив, вычислительно проста, не требует трансцендентных функций и задаётся привычными параметрами H_c , B_r и B_s , но имеет существенный дефект при асимметричных минорных циклах. [1]

Ключевая проблема Chan появляется, когда после разворота поля невозможно продолжить обычную симметричную минорную петлю так, чтобы она пришла в требуемую точку разворота. В такой ситуации Chan использует горизонтальный и вертикальный перенос ветви мажорной петли. Геометрически это может создать ветвь, выходящую наружу относительно мажорной петли. Поскольку наружная ветвь становится слишком полой, симулятор ошибочно воспринимает сердечник как более насыщенный, чем он должен быть.

2.2. Идея улучшения модели гистерезиса Chan

Engelhardt сохраняет простую основу Chan для мажорной петли и обычных симметричных минорных петель, но заменяет проблемный вариант асимметричной ветви. Если симметричную минорную петлю можно корректно продлить до точки разворота, используется обычная вертикально сдвинутая ветвь. Если нельзя, строится новая ветвь как аффинное преобразование специальной базисной функции. Дополнительно эта ветвь ограничивается функцией, связанной с предыдущей минорной ветвью, чтобы она не пересекала ту петлю, из которой произошёл разворот.

3. Математическая основа модели

3.1. Материальные параметры магнитного сердечника

В варианте, приведённом в Table 1 патента, модель использует три основные характеристики материала:

- B_s — saturation flux density, индукция насыщения;
- B_r — remnant flux density, остаточная индукция;
- H_c — coercive force, коэрцитивная напряжённость поля.

В формулах дополнительно присутствует $\mu_0 \cdot H$ — линейный вакуумный вклад.

TABLE 1

Functions used:

```

MIN(x, y) = x < y ? x : y
MAX(x, y) = x > y ? x : y
Upper(H) = Bs * (H + Hc) / (|H + Hc| + Hc * (Bs/Br - 1)) + μ0 * H
Lower(H) = Bs * (H - Hc) / (|H - Hc| + Hc * (Bs/Br - 1)) + μ0 * H
Binit(H) = .5 * (Upper(H) + Lower(H))
UpperBmag(H) = H < 0 ? Lower(H) + Br : Binit(H)
LowerBmag(H) = H > 0 ? Upper(H) - Br : Binit(H)
BlatchUpper(H) = scale * UpperBmag(H) + offset
BlatchLower(H) = scale * LowerBmag(H) + offset
BlatchUpperLimit(H) = Upper(H) - Bd_turn
BlatchLowerLimit(H) = Lower(H) + Bd_turn
Variables used:

```

```

Blast, Hlast, OnInitMag, IsLatched, Bd
Bd_turn, Ho, scale, offset
dir (Direction H used to be changing)

```

3.2. Верхняя и нижняя ветви мажорной петли, и начальная кривая намагничивания

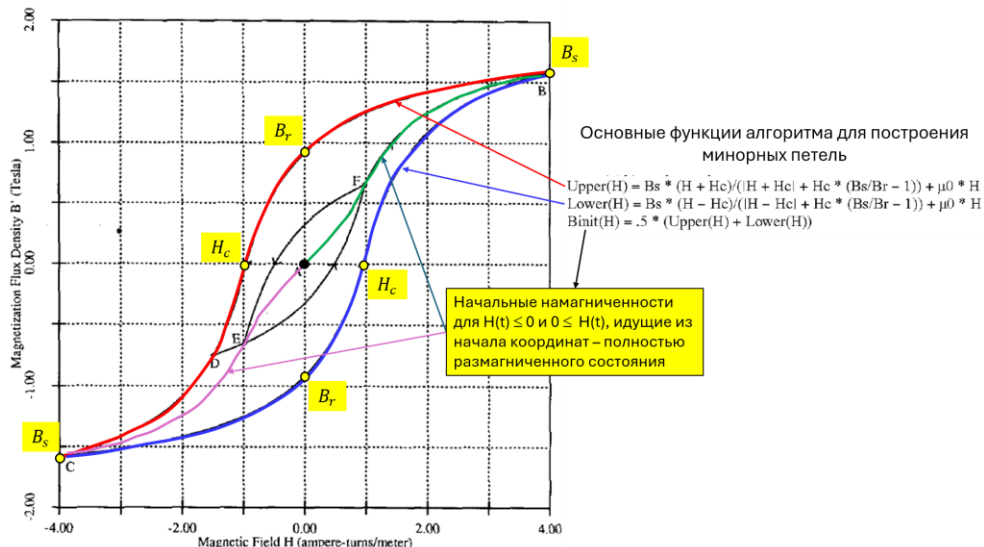
$$\text{Upper}(H) = B_s \cdot (H + H_c) / (|H + H_c| + H_c \cdot (B_s/B_r - 1)) + \mu_0 \cdot H$$

$$\text{Lower}(H) = B_s \cdot (H - H_c) / (|H - H_c| + H_c \cdot (B_s/B_r - 1)) + \mu_0 \cdot H$$

Эти функции из модели Чап задают две границы мажорной петли. Из конструкции следует, в частности, что при $H = 0$ верхняя ветвь проходит через $+B_r$, а нижняя через $-B_r$. Поэтому параметры B_r и H_c встроены в геометрию петли непосредственно, без отдельного численного решения дифференциального уравнения.

$$B_{\text{init}}(H) = \frac{1}{2} \cdot [\text{Upper}(H) + \text{Lower}(H)]$$

Начальная кривая намагничивания задаётся как среднее между ветвями мажорной петли. Для симметричной модели $B_{\text{init}}(0) = 0$. Переменная OnInitMag в алгоритме означает не «модель никогда не разворачивалась», а именно текущее положение траектории на этой начальной кривой.



3.3. Обычные симметричные минорные ветви и параметр Bd

Параметр B_d — вертикальный сдвиг между ветвью минорной петли и соответствующей ветвью мажорной петли. Для участка с возрастающим H используется:

$$B(H) = \text{Lower}(H) + B_d$$

После разворота из движения с уменьшающимся H в возрастающее значение B_d выбирается из условия непрерывности в точке разворота (H_{last} , B_{last}):

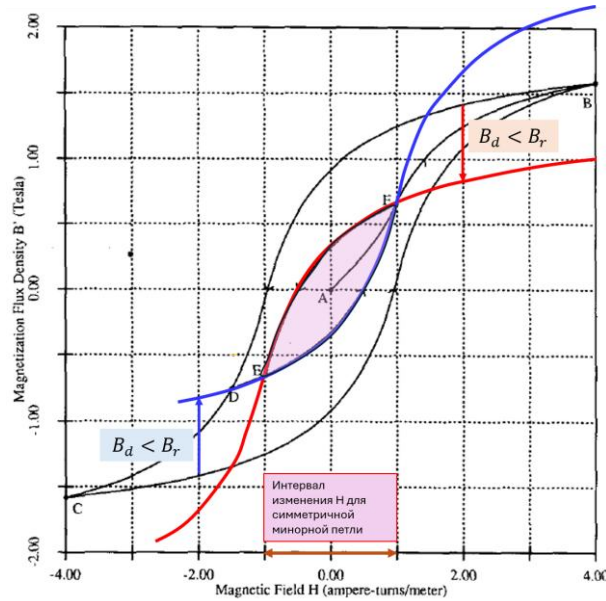
$$B_d = B_{\text{last}} - \text{Lower}(H_{\text{last}})$$

Для участка с уменьшающимся H используется зеркальная формула:

$$B(H) = \text{Upper}(H) - B_d$$

$$B_d = \text{Upper}(H_{\text{last}}) - B_{\text{last}}$$

Эти выражения важны: в точке разворота новая ветвь автоматически проходит через уже вычисленную точку B_{last} и поэтому не создаёт скачка B .



3.4. Базисные функции для аффинного исправления

$$\text{UpperBmag}(H) = \text{Lower}(H) + B_r, H < 0; \quad B_{\text{init}}(H), H \geq 0$$

$$\text{LowerBmag}(H) = \text{Upper}(H) - B_r, H > 0; \quad B_{\text{init}}(H), H \leq 0$$

Это составные базисные кривые. Каждая соединяет начальную кривую намагничивания с ветвью мажорной петли, сдвинутой на B_r . Именно эту форму патент использует как шаблон для проблемной асимметричной минорной ветви.

3.5. Аффинное преобразование

$$\text{BlatchUpper}(H) = \text{scale} * \text{UpperBmag}(H) + \text{offset}$$

$$\text{BlatchLower}(H) = \text{scale} * \text{LowerBmag}(H) + \text{offset}$$

Патент говорит «solve affine map constants gain and offset», но явные алгебраические формулы коэффициентов не выписывает. В нашей реализации они восстанавливаются из требования провести преобразованную базисную кривую через две конечные точки: точку разворота (H_{last} , B_{last}) и точку (H_0 , $B_{\text{init}}(H_0)$). Это уже математическая конкретизация, сделанная в Python-коде, а не дословно приведённая формула патента.

3.6. Ограничивающие функции и смысл `Bd_turn`

$$\text{BlatchUpperLimit}(H) = \text{Upper}(H) - \text{Bd_turn}$$

$$\text{BlatchLowerLimit}(H) = \text{Lower}(H) + \text{Bd_turn}$$

`Bd_turn` — сохранённое значение `Bd` предыдущей ветви в момент разворота. Оно копируется до того, как `Bd` будет пересчитан для новой ветви. Поэтому `Bd_turn` фиксирует геометрию ветви, из которой вышла траектория, и используется как граница, которую новая аффинная ветвь не должна пересекать.

3.7. Точка `H0`

Для возрастающего `H` точка `H0` определяется уравнением:

$$\text{Binit}(H0) = \text{Upper}(H0) - \text{Bd_turn}$$

Для уменьшающегося `H`:

$$\text{Binit}(H0) = \text{Lower}(H0) + \text{Bd_turn}$$

Физически `H0` — поле, при котором ограничивающая ветвь пересекается с начальной кривой намагничивания. До `H0` применяется «latched» аффинное решение. После достижения `H0` latch снимается.

3.8. Формулы `scale` и `offset`, использованные в Python

Для возрастающей ветви используются две точки (`H0`, `B0`) и (`Hlast`, `Blast`), через которые должна пройти кривая `UpperBmag(H)` после аффинного преобразования (масштабирования по высоте и вертикального сдвига):

$$\text{scale} * \text{UpperBmag}(\text{Hlast}) + \text{offset} = \text{Blast}$$

$$\text{scale} * \text{UpperBmag}(H0) + \text{offset} = \text{Binit}(H0)$$

Из этих двух уравнений находим коэффициенты аффинного преобразования:

$$\text{scale} = [\text{Binit}(H0) - \text{Blast}] / [\text{UpperBmag}(H0) - \text{UpperBmag}(\text{Hlast})]$$

$$\text{offset} = \text{Blast} - \text{scale} * \text{UpperBmag}(\text{Hlast})$$

Для уменьшающейся ветви `UpperBmag` заменяется на `LowerBmag`. Эти формулы строго следуют из двухточечной аффинной подгонки и соответствуют формулировке патента о `mapping` базисной функции на конечные точки асимметричной ветви.

4. Анализ рисунков патента

4.1. FIG. 1 — дефект модели Chan

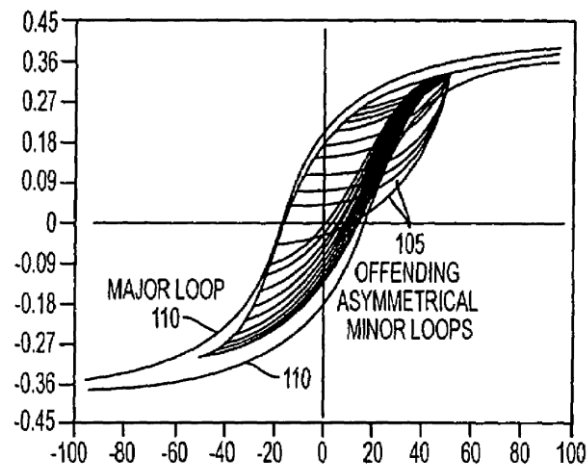


FIG. 1
PRIOR ART

FIG. 1 патента: offending asymmetrical minor loops в модели Chan (prior art).

FIG. 1 иллюстрирует исходную проблему. Мажорная петля обозначена 110, а группа проблемных асимметричных минорных петель — 105. Часть этих ветвей выходит за мажорную петлю. В таком участке наклон dB/dH становится аномально малым, что в circuit simulation эквивалентно ложному впечатлению глубокого насыщения сердечника.

4.2. FIG. 2A — геометрия нормального и проблемного разворота

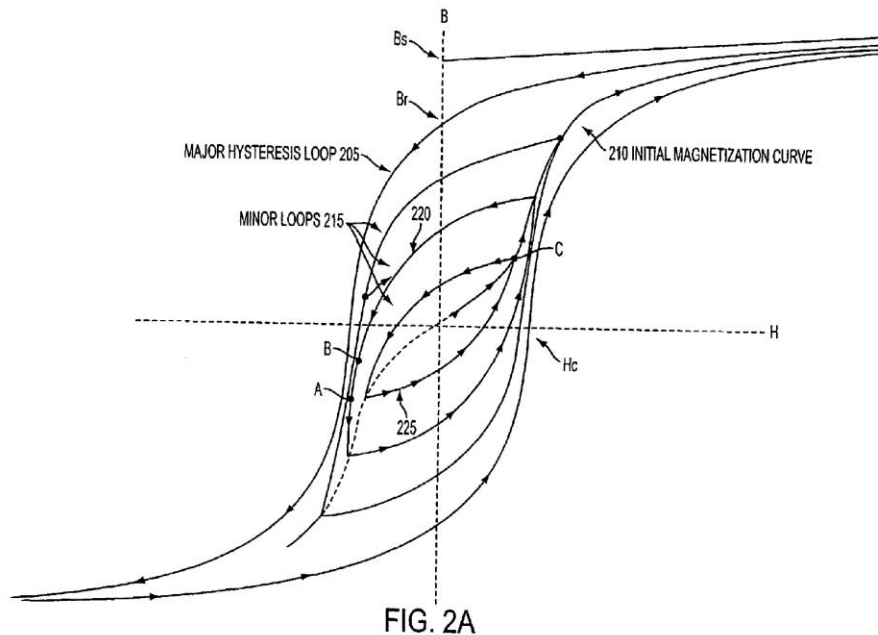
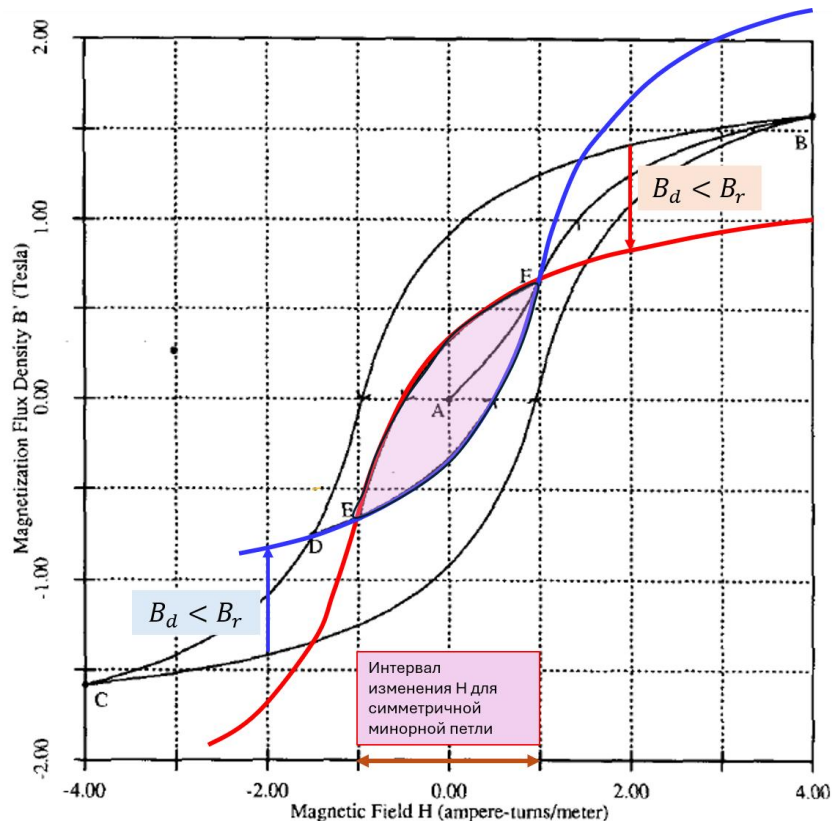
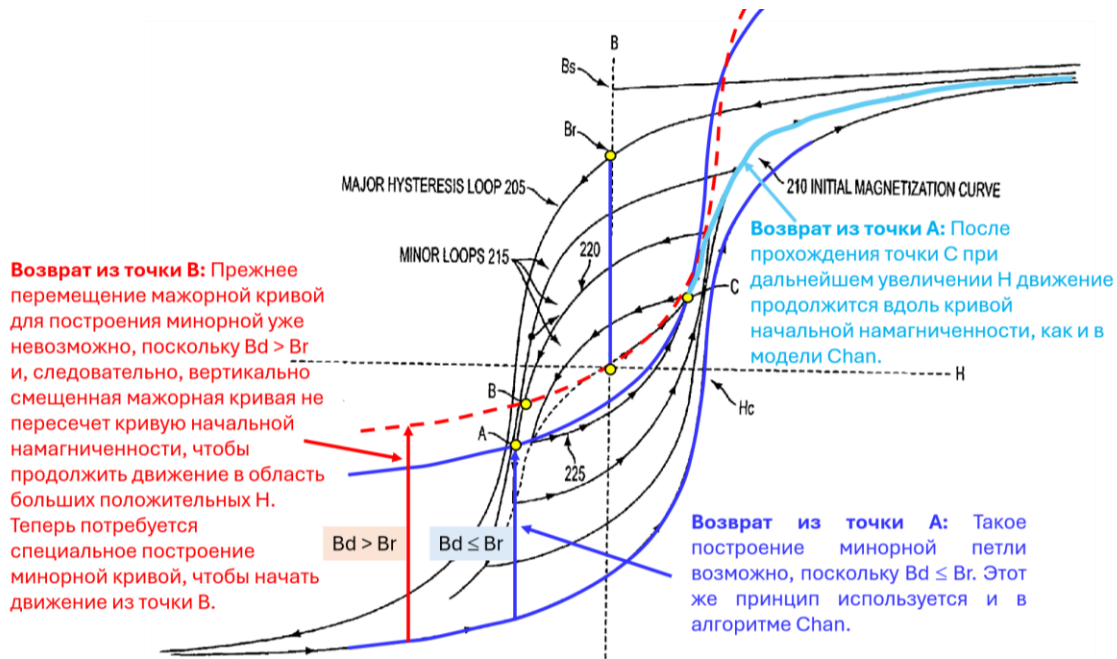


FIG. 2A

FIG. 2A патента: мажорная петля 205, начальная кривая 210, минорные петли 215/220/225 и точки A, B, C.

FIG. 2A — центральный геометрический рисунок патента. Рассматривается движение по верхней ветви минорной петли 220 в сторону уменьшения H . Если разворот происходит в точке А, существует минорная кривая 225, которую можно подтянуть к точке А. Затем траектория в точке С соединяется с initial magnetization curve 210. Патент допускает скачок производной в С и отмечает, что он не создаёт существенной проблемы для симуляции.

Если разворот происходит в точке В, то уже не существует минорной кривой, которую можно было бы подтянуть до В, поскольку это потребовало бы вертикального смещения большего, чем B_r . Именно здесь включается новая конструкция: аффинное отображение базисной функции, ограниченное предыдущей ветвью. Точки А и В поэтому соответствуют двум принципиально разным режимам одного алгоритма: «обычное продолжение» и «affine correction».



4.3. FIG. 2B — результат нового метода

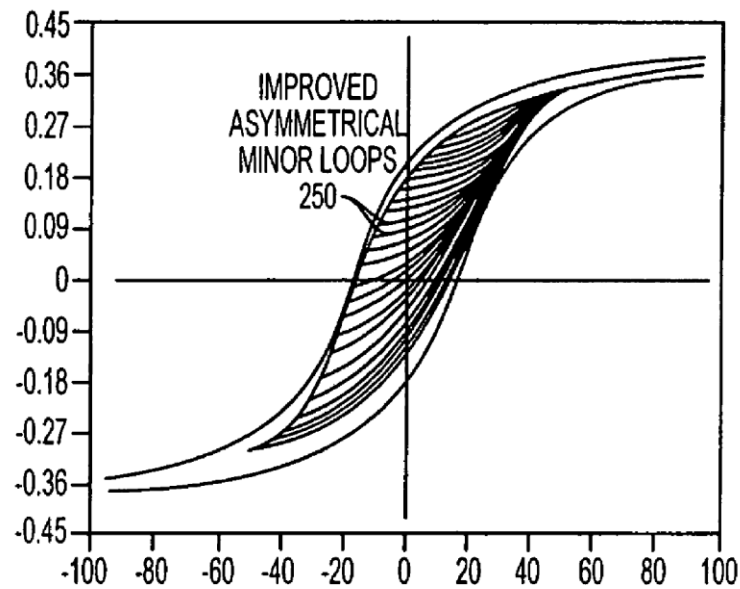


FIG. 2B
NEW METHOD

FIG. 2B патента: improved asymmetrical minor loops 250, построенные новым методом.

FIG. 2B демонстрирует цель патента: семейство асимметричных минорных петель 250 остаётся внутри мажорной петли. Это не означает, что модель воспроизводит всю микрофизику доменной структуры. Её цель — сохранить правдоподобную геометрию $B(H)$ в вычислительно лёгком circuit model.

4.4. FIG. 3A — ветвь алгоритма при возрастающем H

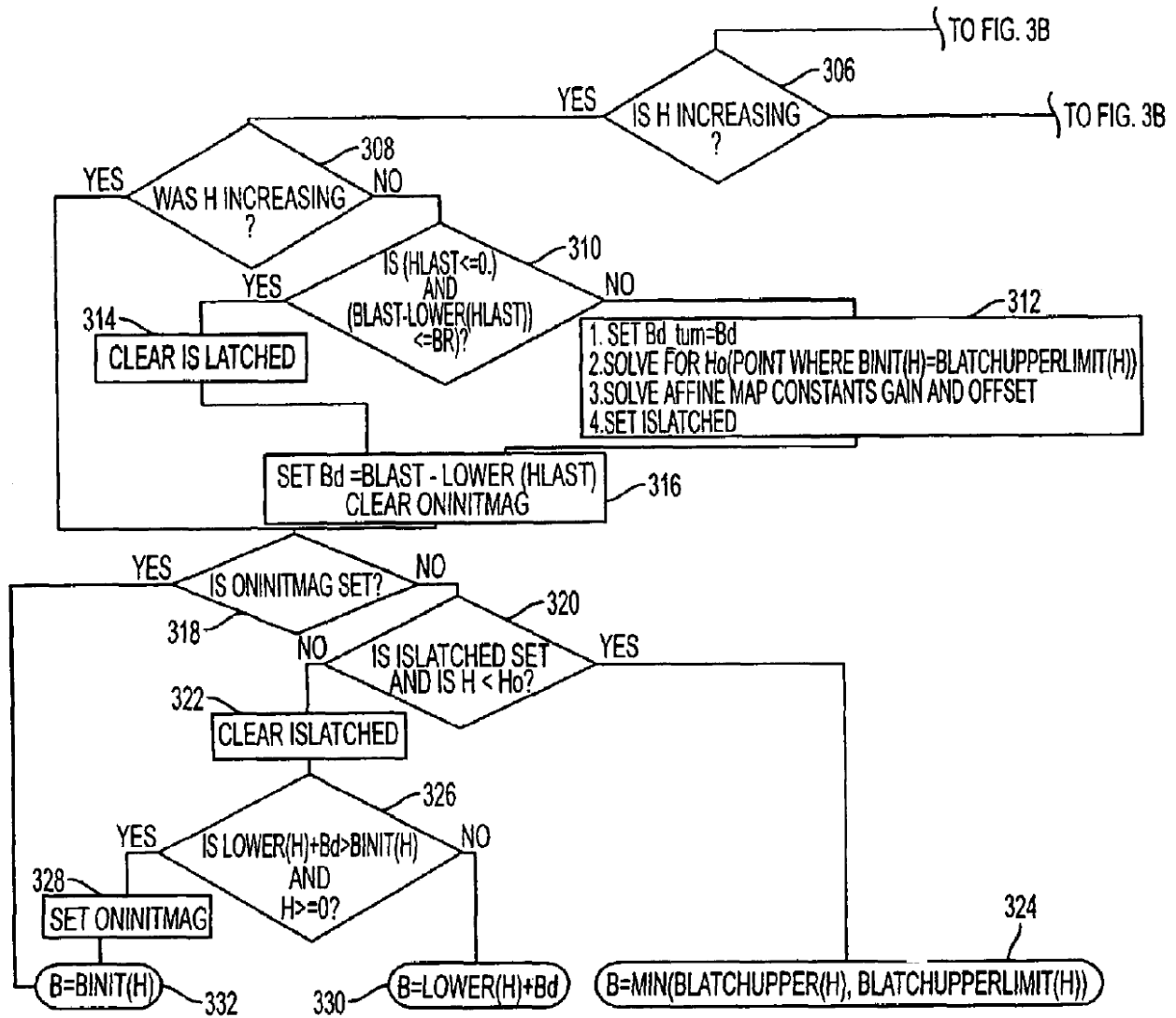


FIG. 3A

FIG. 3A патента: ветвь алгоритма для текущего возрастающего H, шаги 306-332.

FIG. 3A реализует три уровня решения. Сначала определяется, произошёл ли разворот (шаг 308). Затем, если разворот был, проверяется возможность обычной симметричной ветви (шаг 310); при невозможности строится аффинный маппинг (шаг 312). Наконец, шаги 318-332 определяют, какую формулу использовать в текущей точке H: Binit, latched affine branch или обычную Lower(H)+Bd.

4.5. FIG. 3B — ветвь алгоритма при уменьшающемся H

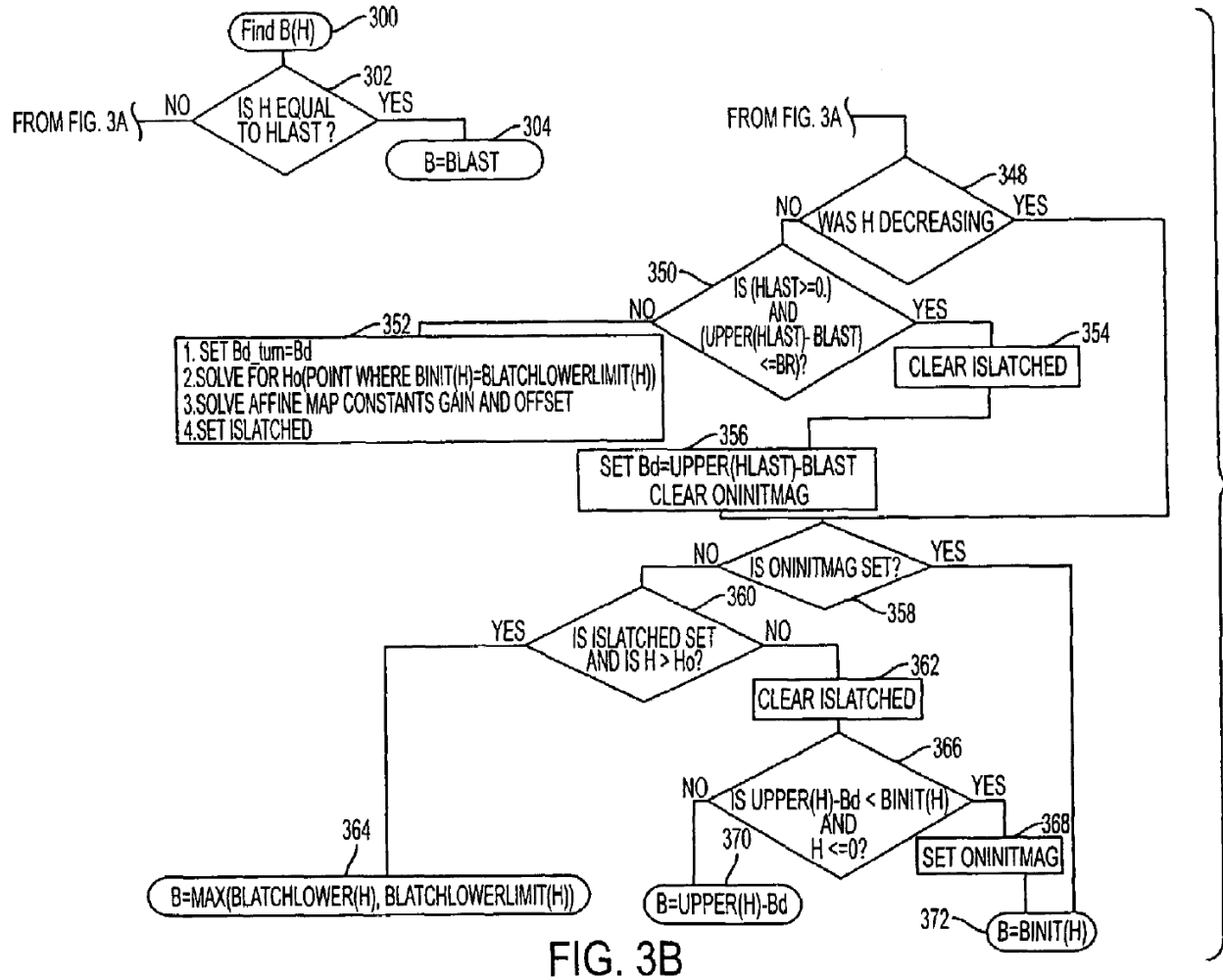
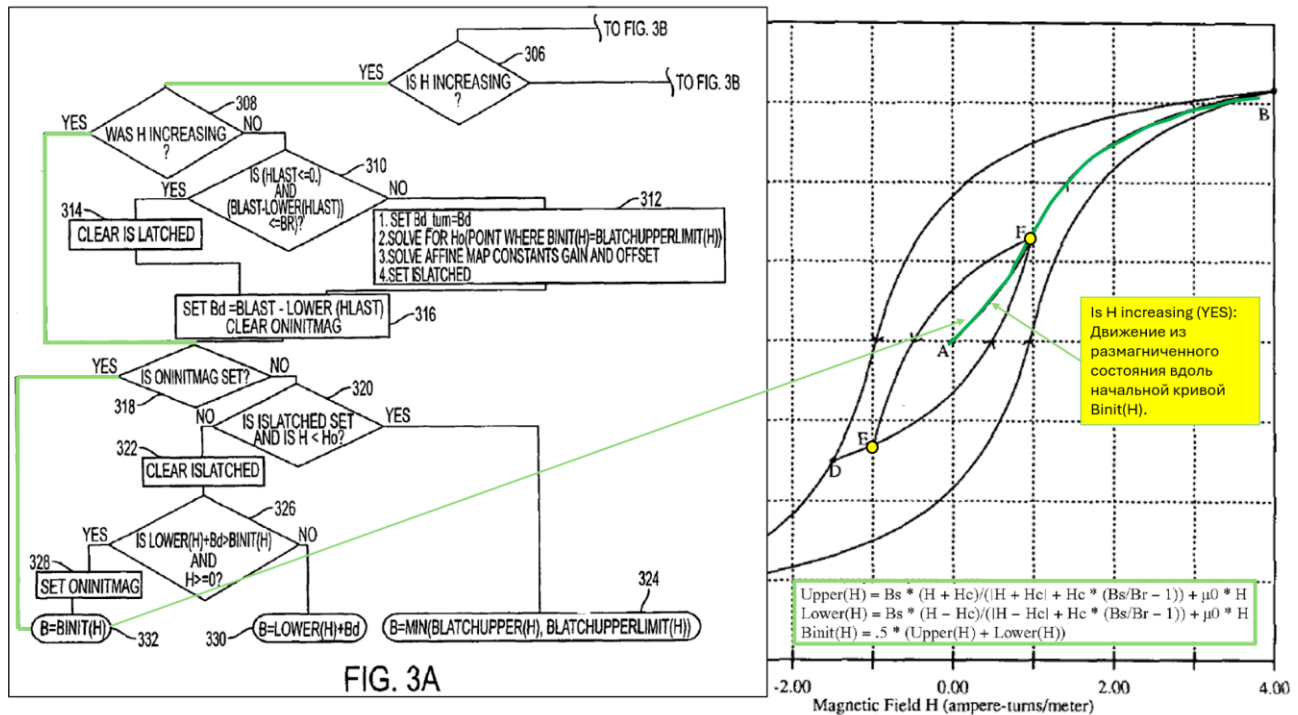


FIG. 3B

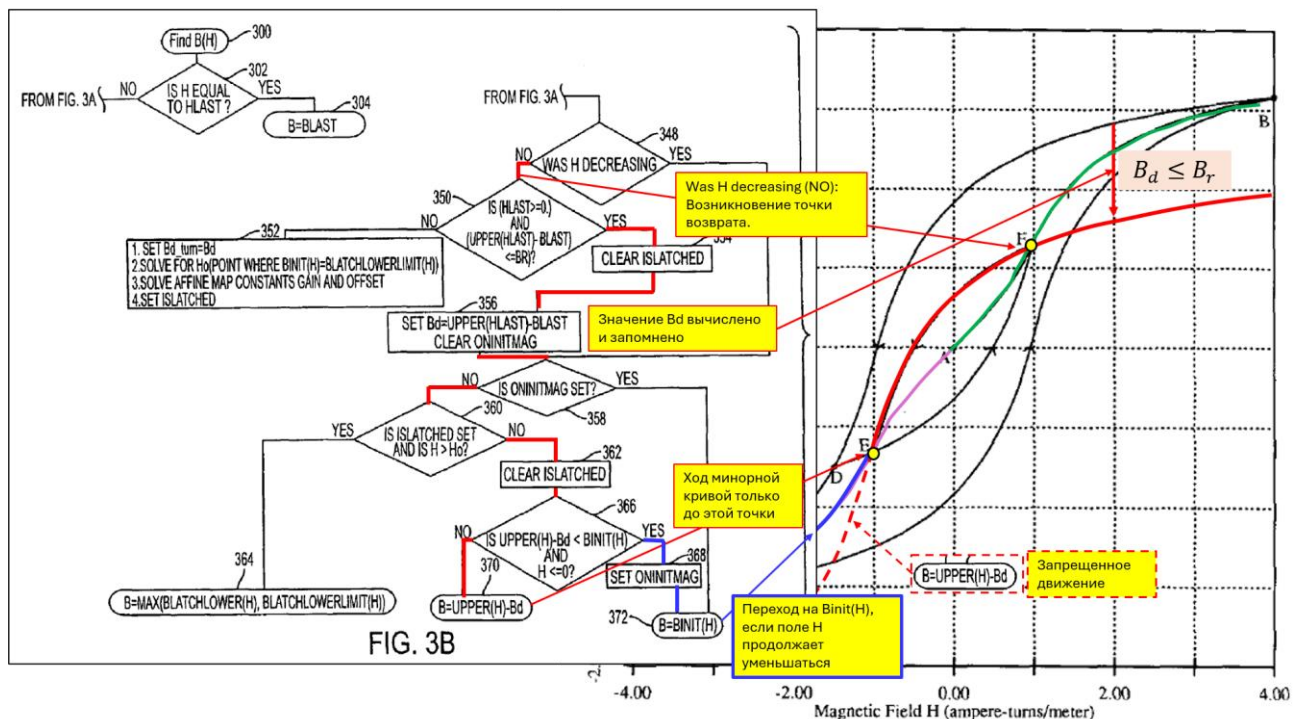
FIG. 3B патента: общая проверка $H = H_{last}$ и зеркальная ветвь для уменьшающегося H , шаги 300-304 и 348-372.

В верхней части FIG. 3B находятся общие шаги 300-304: при $H = H_{last}$ возвращается B_{last} . Основная часть 348-372 является зеркальным дополнением FIG. 3A. При уменьшении поля используются $Upper(H) - B_d$, $BlatchLower$ и $BlatchLowerLimit$, а условие входа на initial magnetization path проверяется в отрицательной области H .

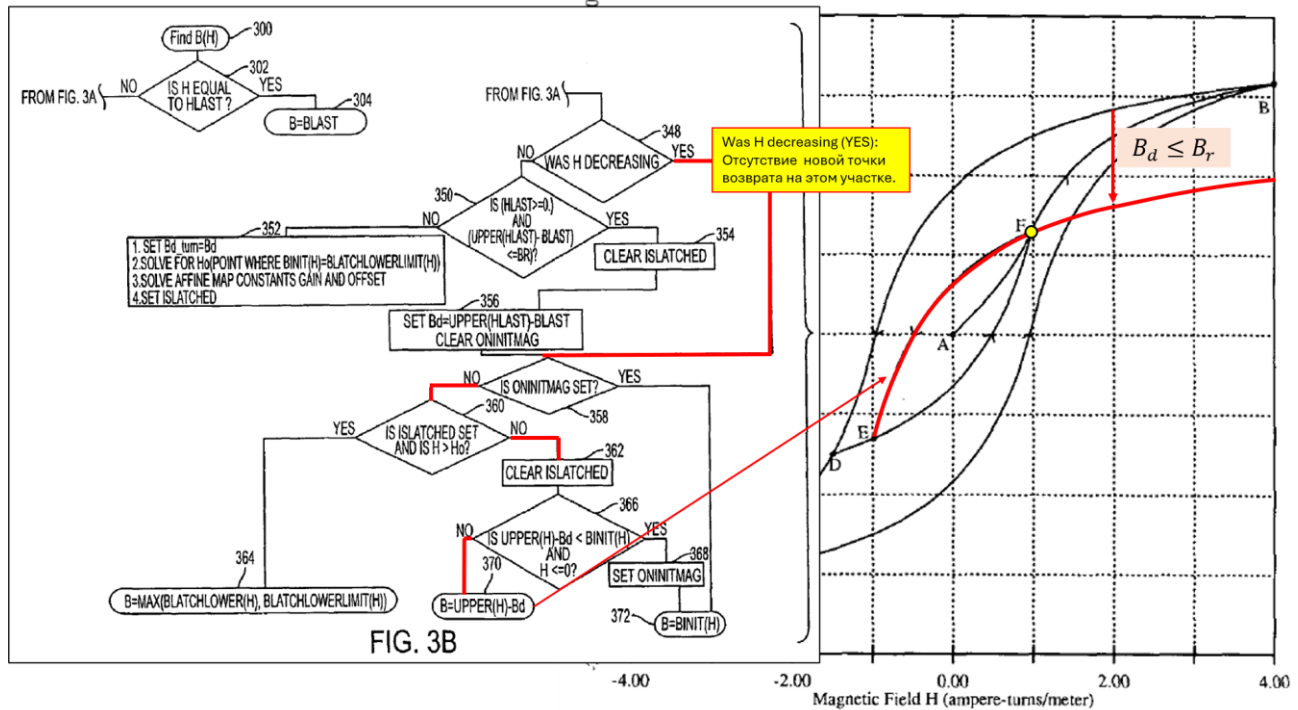
Нижe мы показываем несколько конструкций, показывающих, как двигаться по диаграммам алгоритма в патенте при построении обычных симметричных петель – в этой части они воспроизводят алгоритм Chan. Поскольку нас интересует моделирование эффекта магнитной памяти материала, процесс начинается из полностью размагниченного состояния A до некой точки F на начальной кривой намагничивания.



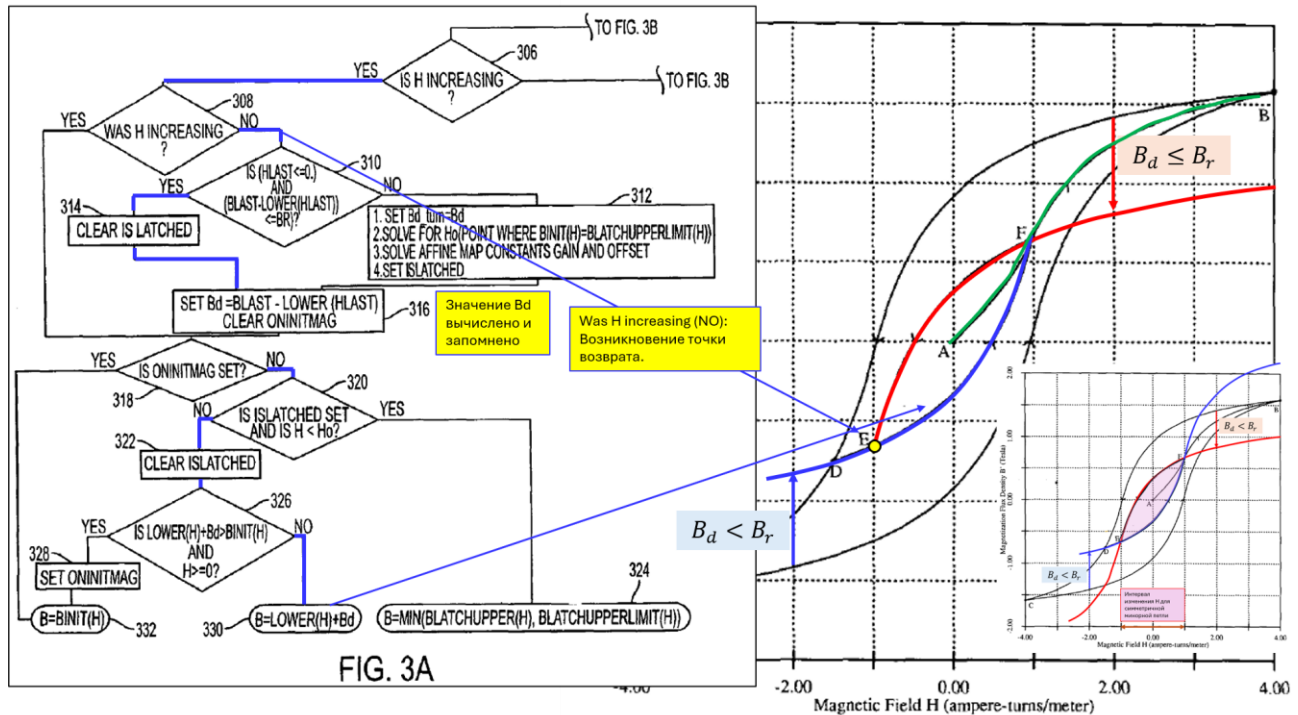
После разворота в точке F движение продолжается вдоль минорной кривой (красная), образованной путем смещения вниз исходной верхней мажорной кривой. После пересечения этой минорной кривой с $B_{\text{init}}(H)$ в точке E движение продолжится вдоль нее (синяя кривая).



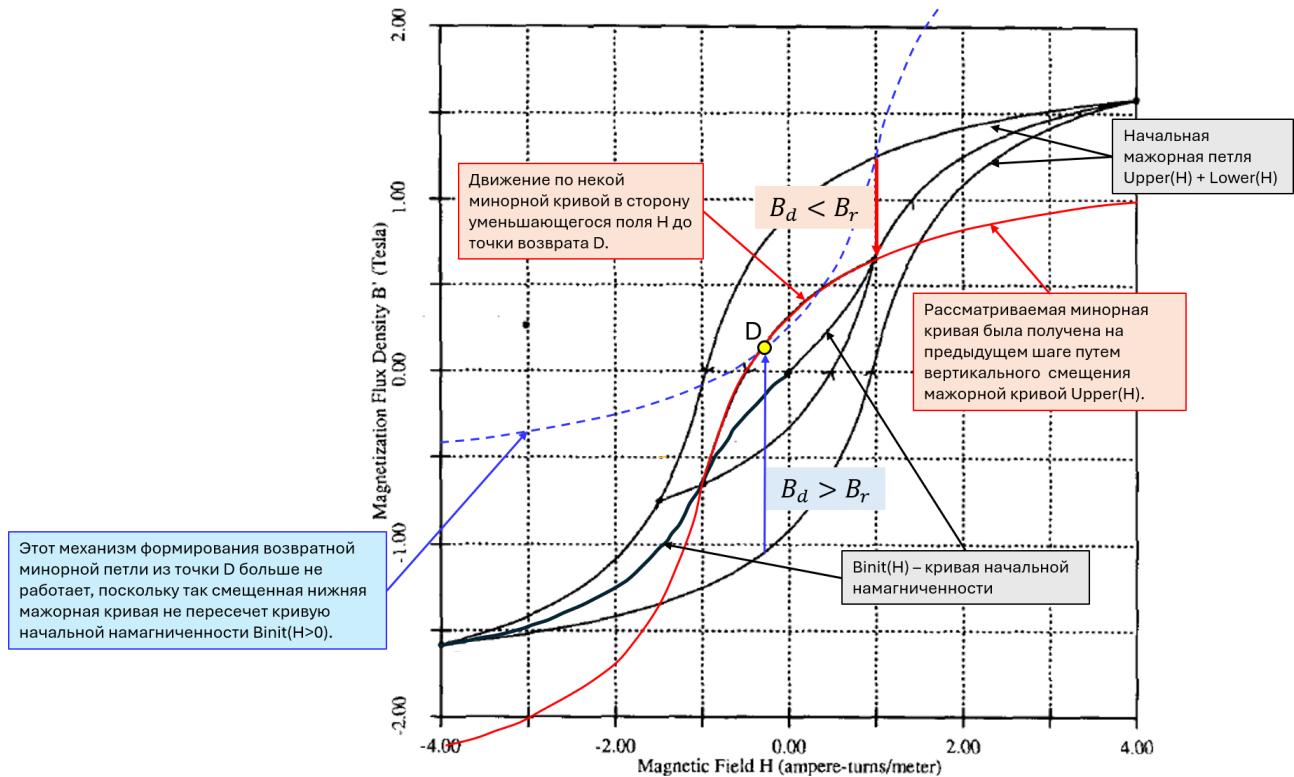
Движение вдоль минорной кривой по убыванию поля без формирования новой точки возврата.



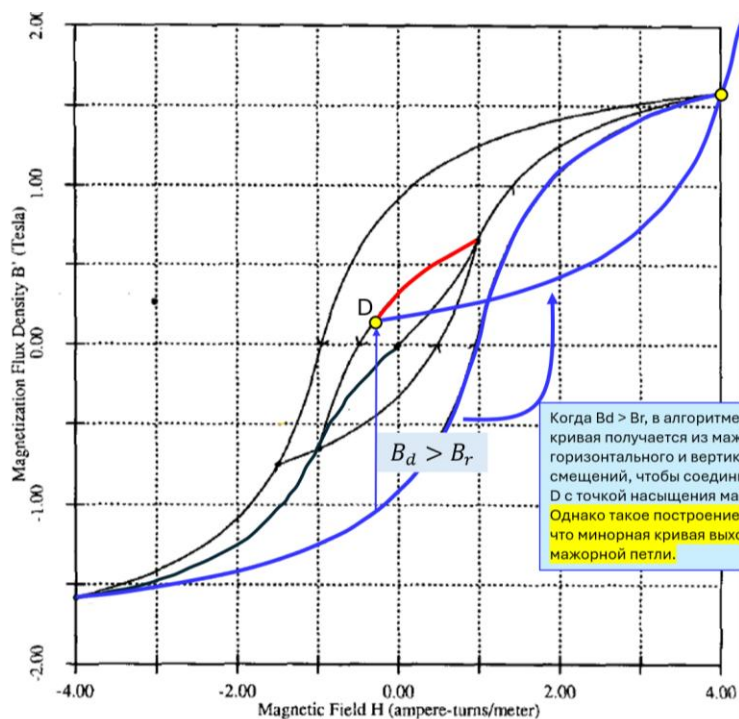
После разворота в точке E движение продолжается вдоль минорной кривой (синяя), образованной путем смещения вверх исходной нижней мажорной кривой. После пересечения этой минорной кривой с Binit(H) в точке F движение продолжится вдоль нее (зеленая кривая). Т.к. обе точки возврата E и F лежат симметрично на начальной кривой Binit(H), то формируется замкнутая симметричная минорная петля (показана розовым цветом на вкладке к рисунку).



Теперь рассмотрим механизм формирования асимметричных минорных петель, когда простое вертикальное смещение мажорных кривых уже невозможно в силу нарушения условия $B_d \leq B_r$, как показано на рисунке ниже. Сначала мы движемся по некоторой минорной кривой до точки возврата D. Однако возвратная кривая из D уже не может быть получена путем $\text{Lower}(H) + B_d$ (вертикального смещения мажорной кривой), т.к. такое смещение будет больше B_r : получающаяся в результате такого смещения минорная кривая (синяя штриховая) не будет пересекаться с кривой начальной намагниченности B_{init} , что не допускается по физическим соображениям.



Метод формирования асимметричной минорной петли в алгоритме Чан, показанный ниже, изначально выглядит довольно нефизичным из-за выхода за пределы мажорной петли. Как продемонстрировано в патенте, при моделировании индуктора с ферромагнитным сердечником, модель Чан может приводить к ненаблюдаемому отклику по току при некоторых режимах возбуждения.



Как объяснено в патенте: На Рис. 1 показаны проблемные асимметричные минорные петли гистерезиса, возникающие в модели Чан. Асимметричные петли гистерезиса могут иметь две формы. Используя обозначения Чан, где B_d — постоянная вертикального смещения между ветвью минорной петли и соответствующей ветвью мажорной петли, если существует минорная петля с $B_d < B_r$, которую можно продолжить за пределы начальной кривой намагничивания до точки разворота, то именно она используется для построения соответствующей ветви асимметричной петли гистерезиса. Если же такой минорной петли не существует, метод Чан предусматривает горизонтальное и вертикальное смещение нижней ветви мажорной петли гистерезиса таким образом, чтобы она проходила через точку разворота. В результате такого смещения возникают проблемные асимметричные минорные петли, демонстрирующие нефизичное поведение: они выходят за пределы мажорной петли гистерезиса.

FIG. 1 illustrates the offending asymmetrical minor hysteresis loops 105 present in Chan's model. Asymmetric hysteresis loops are of two forms. Using Chan's notation of B_d as the constant of vertical translation between a minor loop branch and the corresponding branch of the major loop, if there is a minor loop with B_d less than B_r that can be extended beyond the initial magnetization path to meet the turning point, then that is used for that branch of the asymmetric hysteresis loop. But if there is no such minor loop, then Chan's technique translates the lower major hysteresis loop horizontally and vertically to fit the turning point. This translation results in the offending asymmetrical minor hysteresis loops 105, which exhibit non-physical behavior in that they extend beyond the major hysteresis loop 110. The inductor appears to be saturated when traversing the offending asymmetrical minor hysteresis loops 105 because of the flatness of the slope of these loops. As one skilled in the art will appreciate, circuit simulations or other uses of Chan's model tend to produce erroneous results because the inductor appears to be saturated when it should in fact not be saturated.

В патенте предлагается иной метод формирования асимметричных минорных петель, обсуждаемый ниже, который свободен от недостатков алгоритма Chan. В зависимости от того, по какой минорной кривой мы изначально движемся до аномальной точки возврата:

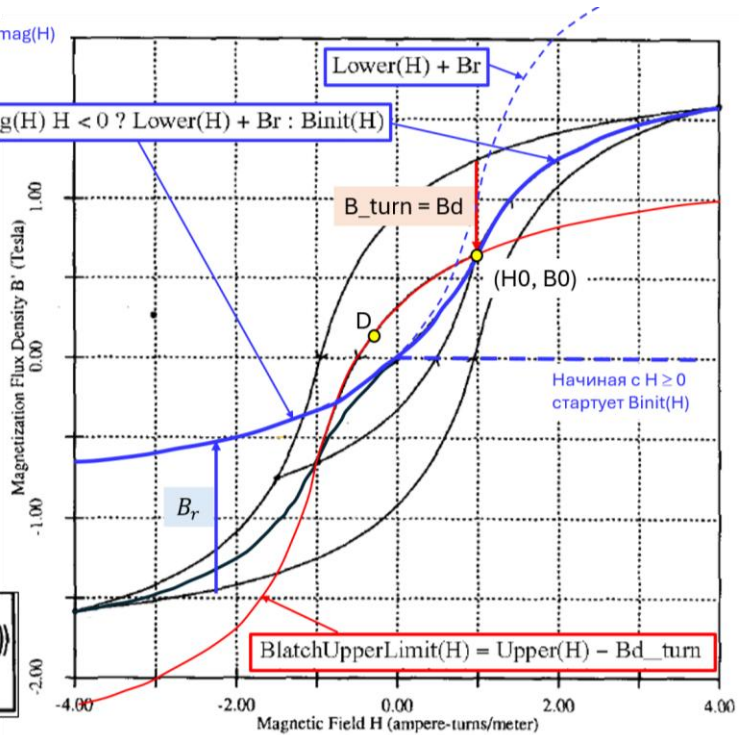
- Находятся координаты точки пересечения (H_0, B_0) кривой $\text{BlatchUpperLimit}(H)$ или $\text{BlatchLowerLimit}(H)$ с кривой намагничивания B_{init} .
- Строится вспомогательная кривая $\text{UpperBmag}(H)$ или $\text{LowerBmag}(H)$.
- Кривая $\text{UpperBmag}(H)$ или $\text{LowerBmag}(H)$ подвергается аффинному преобразованию, совмещающему ее с точками D и (H_0, B_0) . Эта кривая составит возвратный путь до точки (H_0, B_0) , после которой движение продолжится по кривой намагничивания $B_{init}(H)$. В точке сшивания (H_0, B_0) производная может терпеть разрыв, но это не имеет принципиального значения, поскольку при численных расчетах мы не используем производной dB/dH .

Вспомогательная кривая $\text{UpperBmag}(H)$ гладко сшивается из двух:
 $\text{Lower}(H) + B_r$ для $H < 0$
 $B_{init}(H)$ для $H \geq 0$

Шаги 1-4 в алгоритме патента:

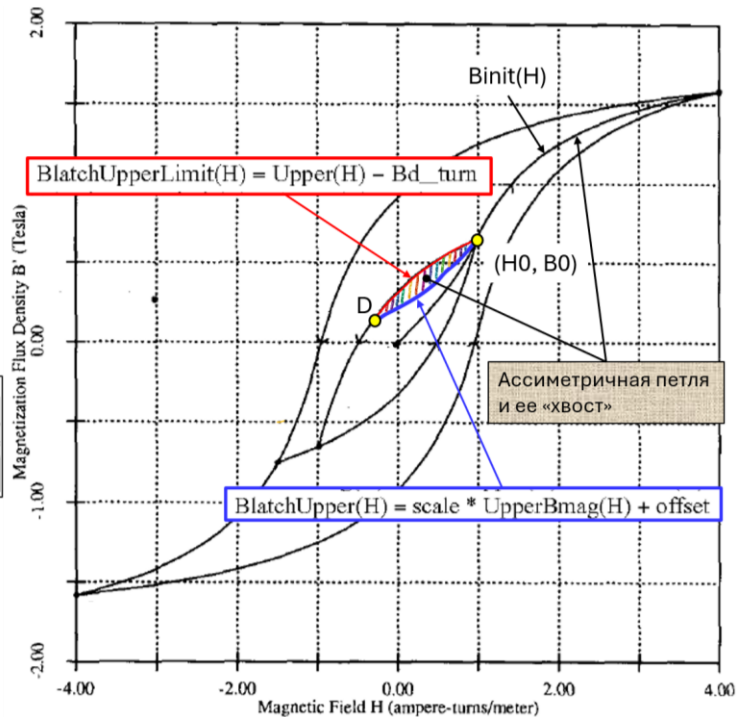
1. Принять $Bd_turn = Bd$, где Bd известно из предыдущего шага.
2. Решить нелинейное уравнение для нахождения координат точки (H_0, B_0) , в которой $B_{init}(H)$ пересекается с $\text{BlatchUpperLimit}(H) = \text{Upper}(H) - Bd_turn$ или $\text{BlatchLowerLimit}(H) = \text{Lower}(H) + Bd_turn$ (в зависимости от минорной кривой).
3. Построить вспомогательную кривую $\text{UpperBmag}(H)$ или $\text{LowerBmag}(H)$, а затем путем аффинного преобразования совместить ее с точками D и (H_0, B_0) . Так преобразованная кривая составит возвратный путь до (H_0, B_0) . Затем движение продолжится по кривой начальной намагниченности $B_{init}(H)$.
4. Выставить флаг $\text{ISLATCHED} = \text{True}$, пока движение происходит между D и (H_0, B_0) . В противном случае $\text{ISLATCHED} = \text{False}$.

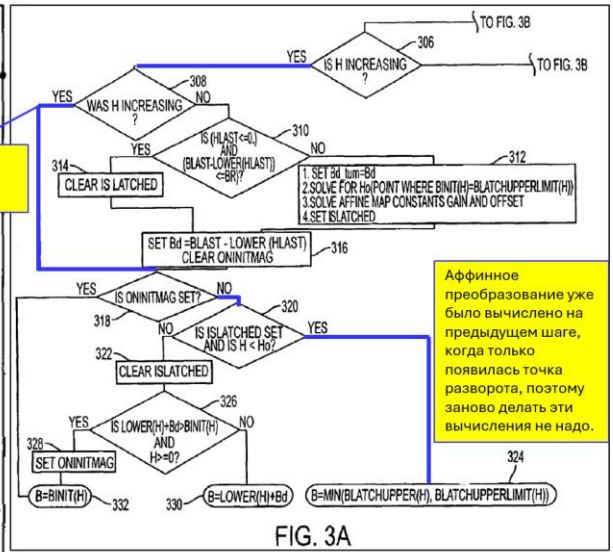
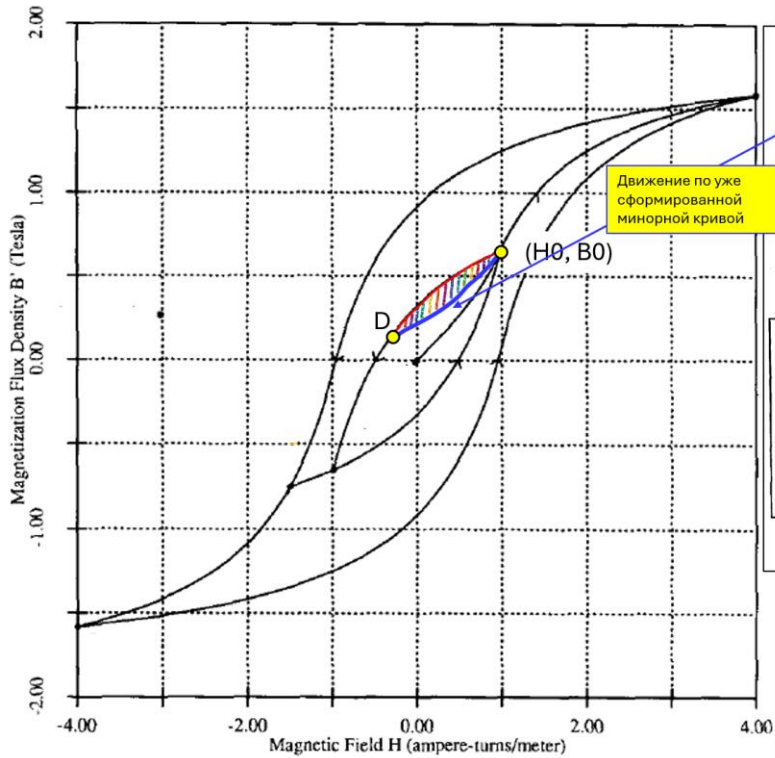
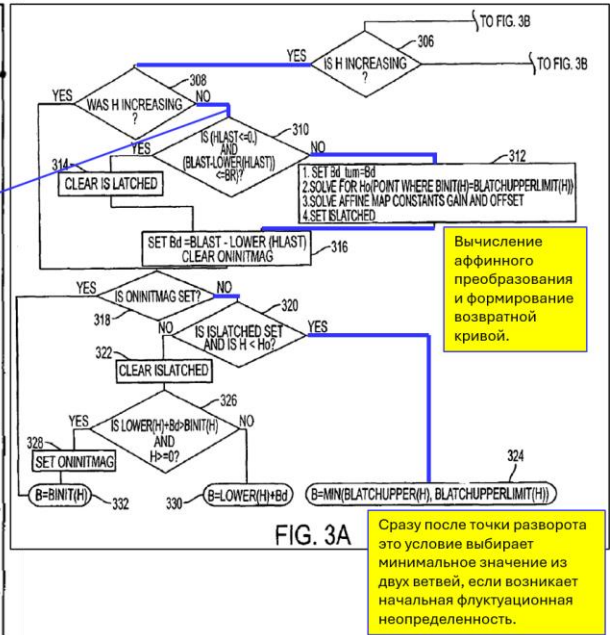
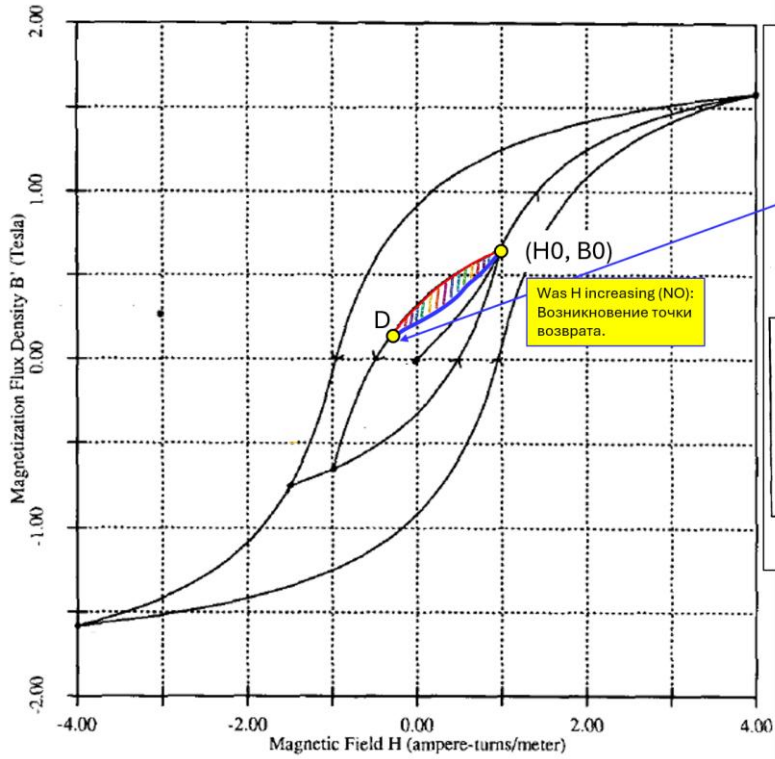
1.SET $Bd_turn=Bd$
 2.SOLVE FOR H_0 (POINT WHERE $B_{init}(H)=\text{BLATCHUPPERLIMIT}(H)$)
 3.SOLVE AFFINE MAP CONSTANTS GAIN AND OFFSET
 4.SET ISLATCHED

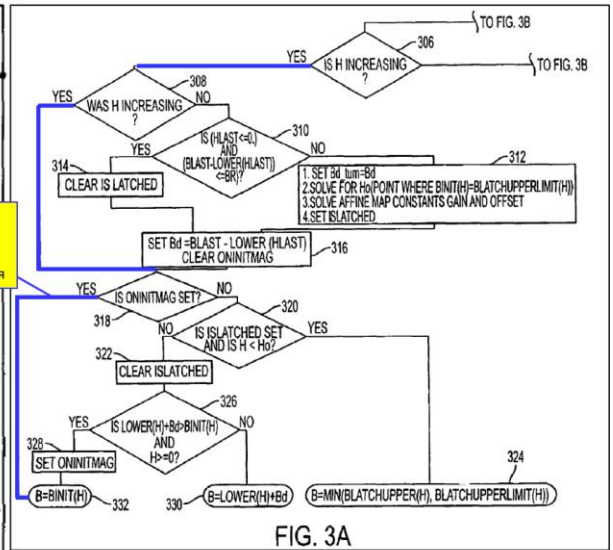
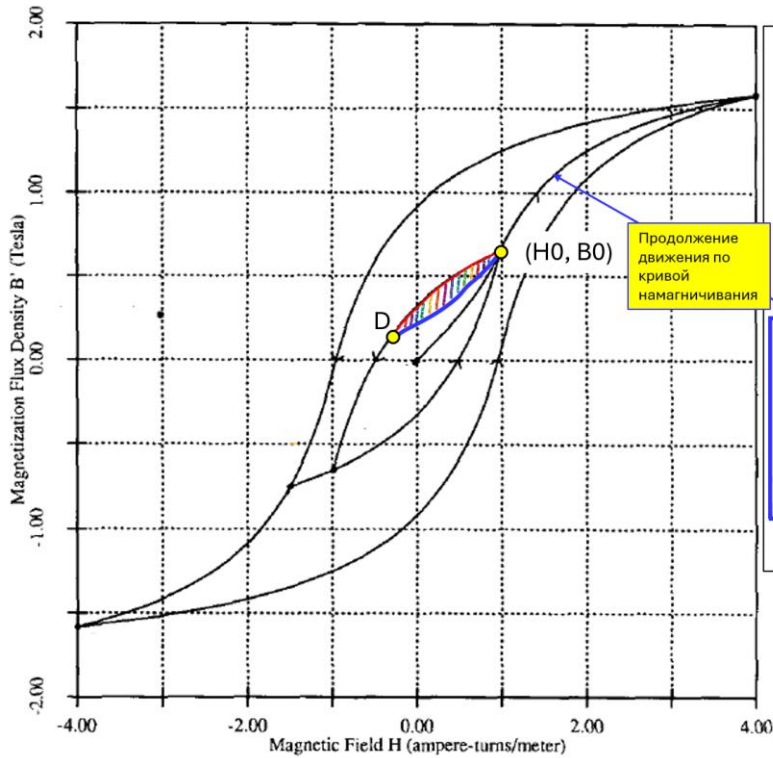
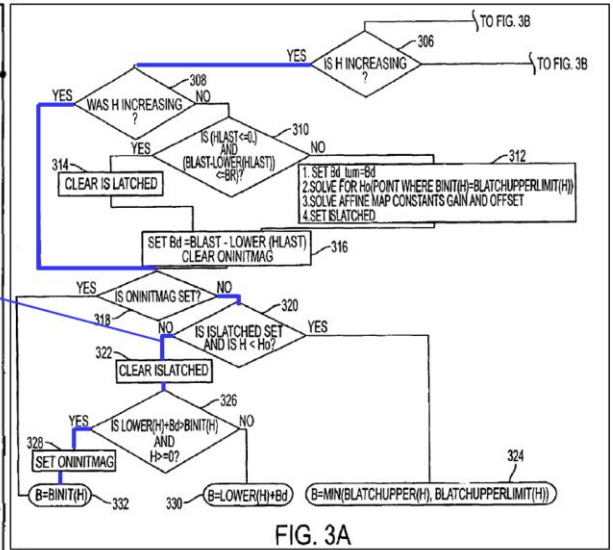
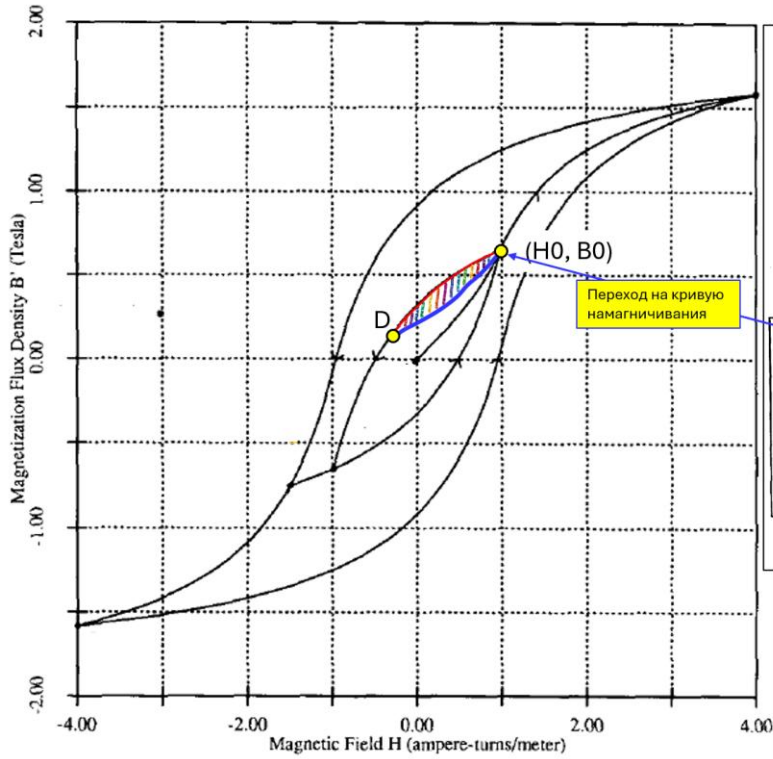


$\text{Upper}(H) = B_s * (H + H_c) / (H + H_c + H_c * (B_s / B_r - 1)) + \mu_0 * H$
 $\text{Lower}(H) = B_s * (H - H_c) / (H - H_c + H_c * (B_s / B_r - 1)) + \mu_0 * H$
 $B_{init}(H) = .5 * (\text{Upper}(H) + \text{Lower}(H))$
 $\text{UpperBmag}(H) \ H < 0 ? \text{Lower}(H) + B_r : B_{init}(H)$
 $\text{LowerBmag}(H) \ H > 0 ? \text{Upper}(H) - B_r : B_{init}(H)$
 $\text{BlatchUpper}(H) = \text{scale} * \text{UpperBmag}(H) + \text{offset}$
 $\text{BlatchLower}(H) = \text{scale} * \text{LowerBmag}(H) + \text{offset}$
 $\text{BlatchUpperLimit}(H) = \text{Upper}(H) - Bd_turn$
 $\text{BlatchLowerLimit}(H) = \text{Lower}(H) + Bd_turn$

1.SET $Bd_turn=Bd$
 2.SOLVE FOR H_0 (POINT WHERE $B_{init}(H)=\text{BLATCHUPPERLIMIT}(H)$)
 3.SOLVE AFFINE MAP CONSTANTS GAIN AND OFFSET
 4.SET ISLATCHED







Модифицированный алгоритм Чап из патента воплощен в популярном электронном симуляторе [LTspice](#) компании [Analog Devices](#) (бывшая [Linear Technology](#)). Мы воспроизвели его на Python с целью дальнейшего использования в проекте по магнитным встряхивателям. Python-алгоритмы обсуждаются ниже.

4.6. FIG. 4 — пример симуляции схемы

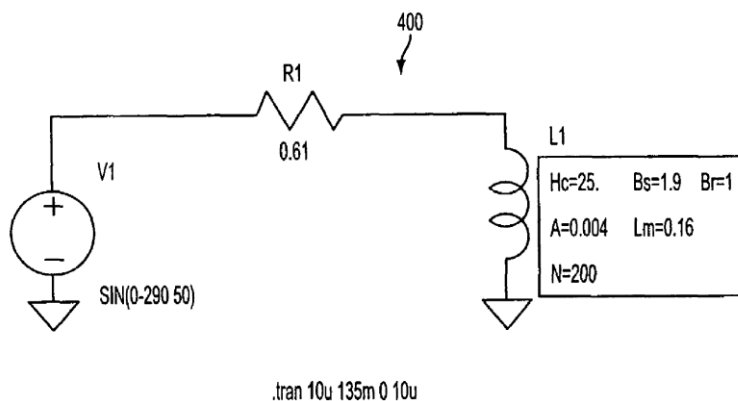


FIG. 4

FIG. 4 патента: тестовая SPICE-схема с источником, $R = 0.61 \Omega$ и нелинейным индуктором.

FIG. 4 показывает не отдельный магнитостатический тест, а включение модели в динамический circuit simulation в LTspice. Синусоидальный источник задан как $\text{SIN}(0 \text{ } -290 \text{ } 50)$, последовательно включён резистор $R1 = 0.61 \Omega$ и индуктор L1. В примере для сердечника указаны $H_c = 25 \text{ A/m}$, $B_s = 1.9 \text{ T}$, $B_r = 1 \text{ T}$, $A = 0.004 \text{ m}^2$ (площадь сечения сердечника), $L_m = 0.16 \text{ m}$ (длина сердечника по средней линии) и $N = 200$ (число витков катушки индуктора). Сердечник не содержит щели, но при необходимости ее можно ввести параметром L_g (длина). При наличии щели, параметр L_m учитывает только длину магнитной части, т.е. общая длина сердечника будет равна $L_m + L_g$. Подразумевается, что сердечник имеет одинаковое сечение по длине.

4.7. FIG. 5A и FIG. 5B — влияние модели на ток индуктора

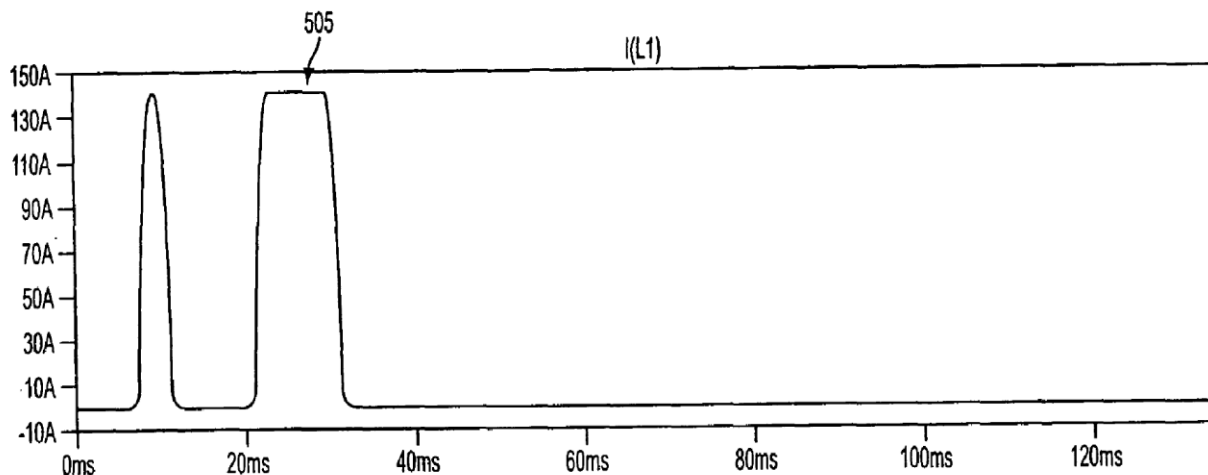


FIG. 5A

FIG. 5A патента: ток $I(L1)$ при prior-art модели; область 505 соответствует ложной чрезмерной насыщенности.

В FIG. 5A ток достигает очень больших значений в области 505, потому что prior-art асимметричная ветвь имеет слишком малый наклон и модель воспринимает сердечник как насыщенный.

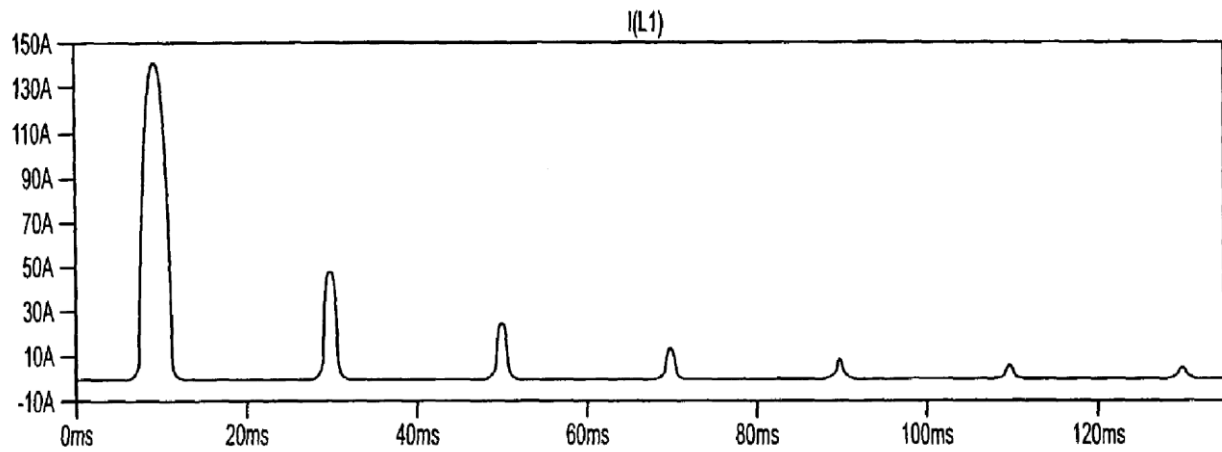


FIG. 5B

FIG. 5B патента: ток $I(L1)$ при новом методе Engelhardt.

FIG. 5B показывает, что после исправления асимметричных ветвей этот эффект существенно подавлен. Для патента это практическое доказательство того, что геометрический дефект $B(H)$ способен заметно исказить уже электрический результат симуляции.

4.8. FIG. 6 — вычислительная реализация

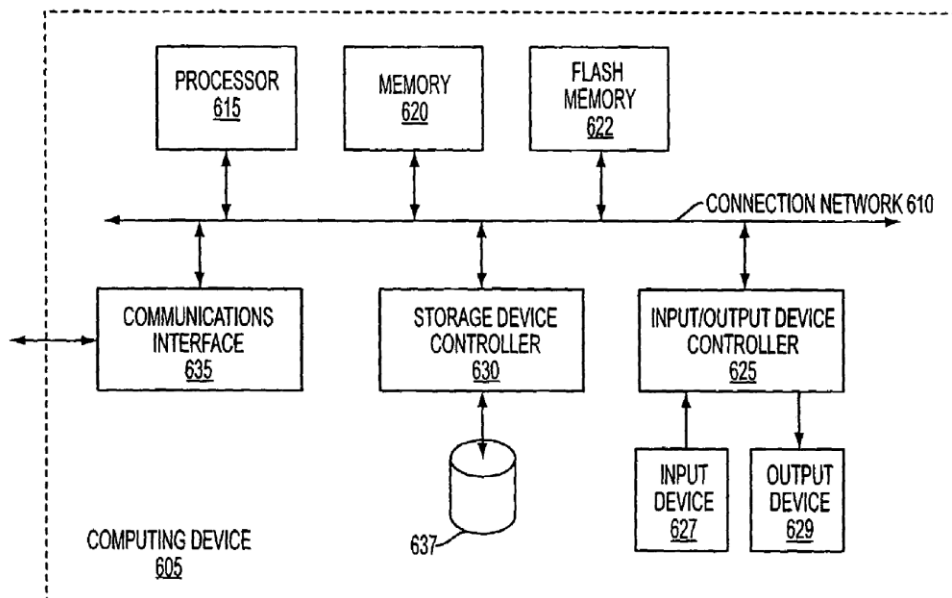


FIG. 6

FIG. 6 патента: обобщённая вычислительная система для выполнения алгоритма.

FIG. 6 не содержит специфической физики гистерезиса; он показывает обычную вычислительную архитектуру: processor, memory, flash memory, I/O controller, storage controller, communications interface и т.д. Его патентная функция — закрепить возможность реализации алгоритма программно, аппаратно или в смешанном виде.

5. Полный алгоритм FIG. 3: шаги 300-372

5.1. Общие шаги 300-306

Шаг	Операция	Смысл
300	Find B(H)	Вход алгоритма: требуется определить B для нового значения H при сохранённой истории.
302	Is H equal to Hlast?	Если поле не изменилось, нет причины пересчитывать ветвь или состояние.
304	B = Blast	Возвращается предыдущее значение B; история не изменяется.
306	Is H increasing?	Выбор одной из двух зеркальных половин: FIG. 3A для $H \uparrow$ или FIG. 3B для $H \downarrow$.

5.2. Возрастающее H: шаги 308-316 — обработка разворота

Шаг 308 спрашивает, увеличивалось ли H и на предыдущем шаге. Если да, текущий участок продолжается без нового разворота. Если нет, точка (Hlast, Blast) является точкой разворота и нужно определить новую ветвь:

Step 310: $(Hlast \leq 0) \text{ AND } (Blast - Lower(Hlast) \leq Br) ?$

Величина $Blast - Lower(Hlast)$ — это именно новый Bd, который будет установлен на шаге 316. Если условие истинно, симметричная минорная ветвь допустима, поэтому шаг 314 просто очищает IsLatched. Если условие ложно, активируется шаг 312 — специальная коррекция.

5.3. Шаг 312 — построение асимметричной ветви для $H \uparrow$

1. SET Bd_turn = Bd. Сохраняется Bd старой ветви, существовавшей до разворота.
2. SOLVE FOR H0. Решается нелинейное уравнение $Binit(H0) = BlatchUpperLimit(H0) = Upper(H0) - Bd_turn$.
3. SOLVE AFFINE MAP CONSTANTS GAIN AND OFFSET. Определяются scale и offset для BlatchUpper.
4. SET IsLatched. Флаг сообщает дальнейшим шагам, что до H0 следует использовать аффинную ветвь.

Затем шаг 316 выполняется независимо от того, был ли выбран шаг 312 или 314:

$Bd = Blast - Lower(Hlast); \text{ OnInitMag} = \text{False}$

Порядок операций принципиален: Bd_turn должен получить старое Bd до того, как Bd будет переопределён для новой возрастающей ветви.

5.4. Шаги 318-332 — выбор формулы при $H \uparrow$

Шаг 318 сначала проверяет OnInitMag. Если он установлен, шаг 332 немедленно даёт $B = Binit(H)$. Иначе рассматривается latch:

Step 320: $IsLatched \text{ AND } H < H0 ?$

Если да, шаг 324 вычисляет:

$B = \text{MIN}[BlatchUpper(H), BlatchUpperLimit(H)]$

MIN — защитная операция: даже если аффинное отображение само пытается пересечь старую петлю, результат остаётся не выше $Upper(H) - Bd_turn$.

Если latch не активен или H достигло/перешло H0, шаг 322 очищает IsLatched. Затем шаг 326 проверяет, пересекла ли обычная ветвь $Lower(H) + Bd$ начальную кривую при $H \geq 0$:

$Lower(H) + Bd > Binit(H) \text{ AND } H \geq 0 ?$

При истинном условии шаг 328 устанавливает OnInitMag, и шаг 332 возвращает $Binit(H)$. В противном случае шаг 330 возвращает обычную минорную ветвь $Lower(H) + Bd$.

5.5. Уменьшающееся Н: шаги 348-356 — зеркальный разворот

Шаг 348 спрашивает, уменьшалось ли Н на предыдущем шаге. Если нет, произошёл разворот из increasing в decreasing:

Step 350: $(H_{last} \geq 0)$ AND $(Upper(H_{last}) - Blast \leq Br)$?

При истинном условии affine correction не нужна, и шаг 354 очищает IsLatched. При ложном условии шаг 352 строит зеркальное affine-решение: $Bd_turn = Bd$, H_0 определяется пересечением Binit с BlatchLowerLimit, затем вычисляются gain/offset и устанавливается IsLatched:

Step 356: $Bd = Upper(H_{last}) - Blast$; $OnInitMag = False$

5.6. Шаги 358-372 — выбор формулы при Н↓

Step 360: IsLatched AND $H > H_0$?

При активном latch используется шаг 364:

$B = MAX[BlatchLower(H), BlatchLowerLimit(H)]$

MAX является зеркальным ограничителем: новая ветвь не может опуститься ниже $Lower(H) + Bd_turn$.

После снятия latch шаг 366 проверяет возвращение на initial magnetization path:

$Upper(H) - Bd < Binit(H)$ AND $H \leq 0$?

Если условие истинно, шаг 368 устанавливает OnInitMag, а шаг 372 возвращает Binit(H). Иначе шаг 370 даёт $B = Upper(H) - Bd$.

Алгоритм удобнее понимать не как набор разрозненных if-условий, а как небольшую машину состояний. В ней геометрическая ветвь определяется не только текущим Н, но и историей.

Переменная	Роль
Hlast, Blast	Последняя вычисленная точка траектории. Нужна для обнаружения разворота и обеспечения непрерывности.
dir / previous_direction	Направление, в котором Н менялось до текущего шага.
OnInitMag	Траектория находится на Binit(H).
IsLatched	Активна специальная аффинная ветвь до точки H_0 .
Bd	Вертикальный сдвиг текущей обычной минорной ветви.
Bd_turn	Замороженное Bd предыдущей ветви в точке разворота; используется как boundary limit.
H_0	Поле окончания latched-режима.
scale, offset	Параметры аффинного преобразования базисной функции.

Таким образом, $B = B(H)$ здесь не является однозначной функцией только текущего Н. Корректнее говорить $B = B(H, state)$, где state хранит память о траектории. Это и есть вычислительная форма гистерезиса.

6. Формулы изобретения (patent claims)

Claims 1-9 организованы в три параллельные группы. Ниже дан технический, а не юридический пересказ.

Формула	Тип	Содержание
1	Компьютерно-реализуемый способ	Верхняя (Upper) и нижняя (Lower) мажорные ветви гистерезиса, кривая Binit как их среднее; симметричные минорные петли как вертикальные сдвиги; асимметричные минорные петли — продолжение симметричной ветви, а при невозможности — ограниченные функции на основе базисных функций; применение в симуляции электронных схем и вывод результата.
2	Уточнение формулы 1	Проблемная асимметричная ветвь является аффинным маппингом функции Bbasis(H), согласованной с точкой возврата и точкой пересечения минорной петли с кривой начальной намагничивания.
3	Уточнение формулы 1	Модель может применяться к сердечнику с зазором.
4-6	Системные формулы	Те же идеи сформулированы как вычислительная система/процессор; формула 5 — аффинный маппинг, формула 6 — цель.
7-9	Машиночитаемый носитель	Те же идеи сформулированы как программный код на носителе; формула 8 — аффинный маппинг, формула 9 — цель.

С инженерной точки зрения наиболее существенная отличительная часть — не сама формула мажорной петли, а способ построения проблемной асимметричной минорной ветви как ограниченный аффинный маппинг базисной функции.

7. Python-проект для расчета петель гистерезиса

Патент Engelhardt можно рассматривать как минимальную stateful-модель гистерезиса, построенную вокруг простой аналитической мажорной петли. Обычные минорные ветви реализуются вертикальными сдвигами Upper/Lower. Новизна и практическая ценность проявляются только в тех разворотах, где обычная симметричная ветвь не может быть продолжена к turning point: тогда создаётся bounded affine map составной basis function. [Текущий Python-проект](#) [4] воспроизводит эту структуру достаточно близко к оригинальной блок-схеме. BH_increasing() и BH_decreasing() являются почти зеркальными транскрипциями FIG. 3A/3B; H0 решается численно; scale и offset вычисляются из endpoint constraints; main.py хранит историю направления и состояния; Hscan.py предоставляет возбуждающие траектории. Следующий качественный этап — не добавление новых формул, а автоматическая верификация инвариантов и создание диагностических тестов, специально заставляющих модель многократно входить в шаги 312 и 352.

После этого модель можно использовать как самостоятельный материал model для расчёта B(H), а затем включать в более широкий динамический расчёт магнитной цепи: потокосцепление, обратная эдс, ток возбуждения, сила и механическое движение актуатора.

7.1 Структура проекта

Python-проект сохранен в папке «Chan-Engelhardt-Model-Symmetrical&Asymmetrical-BHloops» в репозитории [GitHub](#). [4]

```
project/
├── main.py
├── BH.py
├── Hscan.py
├── Parameters.py
└── Parameters.txt
```

Распределение обязанностей:

- Parameters.py — загрузка численных параметров по именам из текстового файла Parameters.txt;
- BH.py — вся физико-алгоритмическая логика B(H): мажорная петля Upper&Lower, средняя кривая намагничивания Binit, базисные функции, H0, аффинный маппинг, возрастающие и убывающие ветви петель BH_increasing/BH_decreasing;
- Hscan.py — формирование истории возбуждающего поля H(t), включая тестовые режимы;
- main.py — дискретный проход по множеству Hvalues, хранение магнитного состояния, вызов функций BH_increasing/BH_decreasing, накопление множества Bvalues и построение графика B(H).

Краткая карта функций и переменных:

Элемент	Файл	Назначение
Upper(H)	BH.py	Верхняя мажорная ветвь.
Lower(H)	BH.py	Нижняя мажорная ветвь.
Binit(H)	BH.py	Кривая начального намагничивания.
UpperBmag(H)	BH.py	Базисная функция для верхнего аффинного маппинга.
LowerBmag(H)	BH.py	Базисная функция для нижнего аффинного маппинга.
BlatchUpper/Lower	BH.py	Аффинно-преобразованные базисные функции.
BlatchUpperLimit/LowerLimit	BH.py	Границы, предотвращающие пересечение предыдущей петли.
solve_H0_upper/lower	BH.py	Локализация корня и метод бисекции для H0.
solve_affine_upper/lower	BH.py	Расчёт коэффициентов scale/offset по условия на конечных точках.
BH_increasing	BH.py	FIG. 3A, current H increasing.
BH_decreasing	BH.py	FIG. 3B, current H decreasing.
Hscan1	Hscan.py	Затухающая синусоида 60 Hz.
Hscan2	Hscan.py	Асимметричное затухающее сканирование.
parameters	Parameters.py	Словарь параметров, прочитанных из файла Parameters.txt.
previous_direction	main.py	Python-аналог патентной переменной dir.

7.2 Модуль функций BH.py

BH.py в начале импортирует словарь parameters и один раз фиксирует Bs, Br и Hc как глобальные переменные модуля. Это означает, что Parameters.txt читается только при импорте Parameters.py, а вызовы Upper(), Lower() и других функций не обращаются к диску:

```
from Parameters import parameters as par

pi = 3.1415926535897932384626433832795
mu0 = 4 * pi * 1.0e-7

Bs = par["Bs"]
Br = par["Br"]
Hc = par["Hc"]
```

Функции мажорной петли и начальной намагниченности буквально повторяют Table 1 патента:

```
def Binit(H):
    return 0.5 * (Upper(H) + Lower(H))

def UpperBmag(H):
    return Lower(H) + Br if H < 0.0 else Binit(H)

def LowerBmag(H):
    return Upper(H) - Br if H > 0.0 else Binit(H)
```

Аналогично для функций BlatchUpper/BlatchLower — реализуют affine map, и BlatchUpperLimit/BlatchLowerLimit — границы, необходимые шаги 324 и 364.

Патент требует «solve for H0», но не предписывает численный метод. В BH.py реализовано двухэтапное решение: сначала корень bracket-ится расширением интервала, затем уточняется методом бисекции. Для increasing-ветви поиск идёт вправо от Hlast, для decreasing — влево. Это разумное и устойчивое решение, поскольку используемые функции непрерывны в рассматриваемой области.

```
def f(H):
    return Binit(H) - BlatchUpperLimit(H, Bd_turn)

# search a bracket to the right of Hlast
# then use bisection until |f(H)| < tol
```

Значение $\text{tol} = 1e-10$ относится к невязке в B, а $\text{max_iter} = 100$ ограничивает как bracket search, так и bisection loop.

Функции `solve_affine_upper()` и `solve_affine_lower()` реализуют двухточечную подгонку. Для upper-функции код формирует $x1 = \text{UpperBmag}(\text{Hlast})$, $y1 = \text{Blast}$, $x2 = \text{UpperBmag}(H0)$, $y2 = \text{Binit}(H0)$, после чего вычисляет slope и intercept. Проверка denominator защищает от вырожденной ситуации, когда две x-координаты basis function совпадают.

Сигнатура функции `BH_increasing()` соответствует шагу 308 «WAS H INCREASING?»:

```
def BH_increasing(
    H, was_inc, Hlast, Blast, OnInitMag, IsLatched,
    Bd, Bd_turn, H0, scale, offset
):
```

Она получает текущее H и полный набор состояния, относящийся к предыдущей точке. Внутри сначала создаются переменные `*_new`, равные старым значениям. Это позволяет менять только те state variables, которые действительно изменяются соответствующим блоком FIG. 3A.

Патентный шаг	Python	Смысл
308	<code>if not was_inc</code>	Разворот из decreasing в increasing.
310	<code>Hlast <= 0 and Blast - Lower(Hlast) <= Br</code>	Проверка, достаточно ли обычной симметричной ветви.
314	<code>IsLatched_new = False</code>	Affine correction не нужна.
312.1	<code>Bd_turn_new = Bd</code>	Сохраняется старый Bd.
312.2	<code>solve_H0_upper(...)</code>	Ищется пересечение Binit с $\text{Upper}(H) - \text{Bd_turn}$.
312.3	<code>solve_affine_upper(...)</code>	Вычисляются scale и offset.
312.4	<code>IsLatched_new = True</code>	Активируется latch.
316	<code>Bd_new = Blast - Lower(Hlast)</code>	Параметр новой increasing-ветви; <code>OnInitMag=False</code> .
318	<code>if OnInitMag_new</code>	Если на initial magnetization path, $B = \text{Binit}$.
320	<code>IsLatched_new and H < H0_new</code>	Проверка, продолжается ли affine segment.
324	<code>min(BlatchUpper, BlatchUpperLimit)</code>	Bounded affine branch.
322	<code>IsLatched_new = False</code>	Latch снят.
326	<code>Lower(H)+Bd_new > Binit(H) and H>=0</code>	Проверка входа на initial path.
328/332	<code>OnInitMag=True; B=Binit(H)</code>	Переход на Binit.
330	<code>B=Lower(H)+Bd_new</code>	Обычная increasing minor branch.

Функция всегда возвращает один и тот же кортеж из восьми элементов: B и семь обновлённых параметров состояния. Такая фиксация порядка важна, поскольку `main.py` распаковывает результат позиционно.

Функция BH_decreasing() специально сделана зеркальной BH_increasing(), что соответствует шагу 348 «WAS H DECREASING?».

Патентный шаг	Python	Смысл
348	if not was_dec	Разворот из increasing в decreasing.
350	Hlast >= 0 and Upper(Hlast) - Blast <= Br	Проверка допустимости симметричной ветви.
354	IsLatched_new = False	Affine correction не нужна.
352.1	Bd_turn_new = Bd	Сохранение старого Bd.
352.2	solve_H0_lower(...)	Binit(H0)=Lower(H0)+Bd_turn.
352.3	solve_affine_lower(...)	Вычисление scale/offset для LowerBmag.
352.4	IsLatched_new = True	Активирование latch.
356	Bd_new = Upper(Hlast) - Blast	Параметр новой decreasing-ветви; OnInitMag=False.
358	if OnInitMag_new	B=Binit(H).
360	IsLatched_new and H > H0_new	Affine segment ещё активен.
364	max(BlatchLower, BlatchLowerLimit)	Bounded affine branch.
362	IsLatched_new = False	Latch снят.
366	Upper(H)-Bd_new < Binit(H) and H<=0	Проверка входа на initial path.
368/372	OnInitMag=True; B=Binit(H)	Переход на Binit.
370	B=Upper(H)-Bd_new	Обычная decreasing minor branch.

Использование min на upper-latched branch и max на lower-latched branch — не косметическая симметрия, а геометрическая защита от пересечения предыдущей петли.

7.3 Управляющий цикл main.py

Модуль main.py выполняет роль верхнего уровня FIG. 3: генерирует H(t), хранит history state, определяет направление движения и вызывает соответствующую половину алгоритма.

Внешние вызовы:

```
import numpy as np
import matplotlib.pyplot as plt
from BH import BH_increasing, BH_decreasing
from Hscan import Hscan1, Hscan2
```

Инициализация:

```
Hlast = 0.0
Blast = 0.0
OnInitMag = True
IsLatched = False
Bd = 0.0
Bd_turn = 0.0
H0 = 0.0
scale = 0.0
offset = 0.0
```

Такой старт соответствует размагниченному состоянию $(H, B) = (0, 0)$ на initial magnetization path. Проверка Hvalues[0] ≈ 0 является важной предпосылкой этой инициализации.

Переменная previous_direction может принимать значения "inc", "dec" или None. На первой ненулевой итерации соответствующий was_inc/was_dec устанавливается True, чтобы старт из (0,0) не интерпретировался как разворот. После каждого реального шага previous_direction обновляется.

Программные аналоговые шаги 302-304:

```
if np.isclose(H, Hlast):
    B = Blast
    # previous_direction is NOT changed
```

Состояние сохраняется, а направление не меняется. Такое поведение особенно важно в окрестности экстремума тестового $H(t)$, где дискретизация может давать одинаковые или почти одинаковые отсчёты.

Множество значений (arrays) Bvalues имеет ту же длину, что Hvalues. Изначально Bvalues[0]=0. После каждой итерации текущая точка записывается, затем Hlast = H и Blast = B. В конце создаётся массив BHloop = column_stack((Hvalues, Bvalues)), после чего библиотека matplotlib строит траекторию в порядке времени, а не сортирует точки по H. Это принципиально: гистерезис исчез бы, если отсортировать точки по H.

7.4 Тестовые функции из модуля Hscan.py

Тестовую функцию надо выбрать в main.py:

```
from Hscan import Hscan1, Hscan2
```

$$Hscan1(t) = 600 \sin(120\pi t) \exp[-t/(3T)], \quad T = 2\pi/(120\pi) = 1/60 \text{ s}$$

Таким образом, Hscan1 — синусоидальное поле 60 Hz с начальной амплитудой 600 A/m и экспоненциальной постоянной затухания $3T = 0.05 \text{ s}$. Такой сигнал естественно создаёт последовательность уменьшающихся по амплитуде петель вокруг нуля и полезен для общей проверки memory behavior.

$$Hscan2(t) = Hbias(1 - e^{-(t/\tau_{bias})}) + A0 e^{-(t/\tau_{decay})} \sin(\omega t)$$

В текущем коде Hbias = 0.8*Hc, A0 = 1.5*Hc, n_cycles = 5, $\tau_{bias} = 0.01 \cdot T_{sim}$, $\tau_{decay} = 0.8 \cdot T_{sim}$, где T_{sim} — время симуляций. Цель Hscan2 — создать несколько разворотов вокруг положительного смещения и тем самым чаще активировать affine-correction branches.

7.5 Загрузка параметров в Parameters.py

Parameters.py один раз открывает Parameters.txt при импорте, пропускает пустые строки и комментарии, делит каждую строку по первому символу "=" и сохраняет float-значение в dictionary parameters.

```
parameters = {}

with open(file_path, "r", encoding="utf-8") as f:
    for line in f:
        line = line.strip()
        if not line or line.startswith("#"):
            continue
        name, value = line.split("=", 1)
        parameters[name.strip()] = float(value.strip())
```

В BH.py затем выполняется `from Parameters import parameters as par`, а Bs, Br, Hc считываются из словаря один раз. Поэтому повторного чтения текстового файла при каждом вызове B(H) нет. Одновременно это означает, что изменение Parameters.txt во время уже работающего процесса автоматически не обновит Bs, Br и Hc в BH.py.

7.6 Соответствие патента и Python-реализации

Элемент	Патент	Текущая реализация
Формулы Upper/Lower/Binit	Буквально Table 1	Прямое соответствие.
UpperBmag/LowerBmag	Буквально Table 1	Прямое соответствие.
BlatchUpper/Lower и limits	Буквально Table 1	Прямое соответствие.
Условия шагов 310/350, 320/360, 326/366	Буквально FIG. 3	Перенесены в if-условия.
Bd при развороте	Буквально шаги 316/356	Прямое соответствие.
Сохранение Bd_turn	Буквально шаги 312/352	Прямое соответствие.
Численный метод поиска H0	Не задан	Выбран bracket + bisection.
Формулы scale/offset	Не выписаны	Выведены из двух endpoint conditions.
Representation of dir	Патент: dir	Python: previous_direction + was_inc/was_dec.
Общий wrapper Find B(H)	В патенте шаг 300	В текущем проекте разделён между main.py и двумя ВН-функциями.

Особенно важно не приписывать патенту явные формулы scale и offset: патент требует их «solve», но итоговые выражения в отчёте и коде являются нашей алгебраической реализацией этого требования.

7.7 Проверки, тесты и диагностические инварианты

Ниже приведены проверки, которые желательно автоматизировать.

Проверка	Критерий	Что обнаруживает
Базовые точки	$Upper(0) \approx Br$; $Lower(0) \approx -Br$; $Binit(0) \approx 0$	Проверяет параметризацию мажорной петли.
Continuity at reversal, $H \uparrow$	$Lower(H_{last}) + Bd_{new} = Blast$	Новая ветвь проходит через turning point.
Continuity at reversal, $H \downarrow$	$Upper(H_{last}) - Bd_{new} = Blast$	Зеркальная проверка.
H0 residual, upper	$Binit(H0) - BlatchUpperLimit(H0) \approx 0$	Проверка root solver.
H0 residual, lower	$Binit(H0) - BlatchLowerLimit(H0) \approx 0$	Проверка root solver.
Affine endpoint upper	$BlatchUpper(H_{last}) = Blast$ и $BlatchUpper(H0) = Binit(H0)$	Проверка scale/offset.
Affine endpoint lower	$BlatchLower(H_{last}) = Blast$ и $BlatchLower(H0) = Binit(H0)$	Проверка scale/offset.
Major-loop enclosure	$Lower(H) \leq B \leq Upper(H)$	Главный тест отсутствия Chan-type артефакта.
Latch upper bound	$B \leq BlatchUpperLimit(H)$ при upper latch	Не пересекать старую ветвь.
Latch lower bound	$B \geq BlatchLowerLimit(H)$ при lower latch	Зеркально.
$H = H_{last}$	$B = Blast$, state не меняется	Шаги 302-304.
Sign symmetry	При симметричных параметрах ответ на $-H$ -history должен быть зеркален	Сильный regression test.

Автоматический контроль выхода за мажорную петлю:

```

Bupper = Upper(Hcalc)
Blower = Lower(Hcalc)

outside = (Bcalc > Bupper + tol) | (Bcalc < Blower - tol)

if np.any(outside):
    raise RuntimeError("B-H trajectory left the major loop")

```

Именно этот invariant непосредственно проверяет дефект Chan, ради устранения которого и создавался патент.

7.8 Замечания к текущей реализации и рекомендации

Точность `np.isclose` для определения неподвижного H:

В `main.py` используется `np.isclose(H, Hlast)` с default `rtol` и `atol`. Для определения направления поля лучше задать собственный абсолютный допуск, например ``np.isclose(H, Hlast, rtol=0.0, atol=H_tol)``, чтобы относительный допуск не зависел от величины H и случайно не «съедал» маленькие шаги около экстремума.

Начальные значения `H0`, `scale`, `offset` и `Bd_turn`:

В текущем `main.py` они инициализируются нулями. Это допустимо, потому что при `IsLatched=False` соответствующие значения не должны использоваться. Для отладки более строгим вариантом было бы `None`, поскольку тогда случайное использование неинициализированного `latch`-параметра выявится сразу. Однако существующая архитектура уже содержит проверки `None` внутри `BH.py`, поэтому переход к `None` может быть сделан позже.

Edge case `H0 = Hlast`:

`solve_H0_upper/lower` немедленно возвращают `Hlast`, если `f(Hlast)` уже близка к нулю. После этого `solve_affine_*` может получить почти нулевой `denominator`, если `basis function` в `Hlast` и `H0` совпадает. Это не обязательно ошибка — геометрически это вырожденный `affine segment` — но такой случай следует покрыть отдельным тестом и решить, как его трактовать: как отсутствие необходимости `latch`, как нулевую длину `segment` или как допустимую особую ветвь.

Ограничение поиска `H0`:

`Bracketing` удваивает шаг до нахождения смены знака или исчерпания `max_iter`. Для `production`-кода полезно ограничить физически разумный диапазон H и включить в диагностическое сообщение `Hlast`, `Bd_turn` и значения `f` на концах интервала. Это заметно упростит поиск ошибок состояния.

Явный `top-level BH_step()`:

Патент концептуально имеет одну операцию `Find B(H)`, а текущая программа разделяет её между `main.py` и `BH_increasing/BH_decreasing`. Для дальнейшего развития удобно ввести `wrapper BH_step(H, state)`, который сам выполняет проверку `H≈Hlast`, `direction detection` и вызов нужной ветви. Тогда `main.py` будет отвечать только за `time scan`, `storage` и `plotting`.

7.9 Результаты расчета

Программа рисует график гистерезиса для выбранной тестовой функции $H(t)$ из модуля Hscan.py, начиная с полностью размагниченного состояния, и сохраняет его в Excel файле BHloop.csv. В этом модуле можно создать и сохранить любую тестовую функцию.

```
# =====
# DRAW B(H)
# =====

plt.figure(figsize=(7, 6))

plt.plot(
    BHloop[:, 0],
    BHloop[:, 1],
    linewidth=1.5
)

plt.xlabel('H (A/m)')
plt.ylabel('B (T)')
plt.title('B-H Hysteresis Loop')

plt.grid(True)

plt.axhline(0.0, linewidth=0.8)
plt.axvline(0.0, linewidth=0.8)

plt.tight_layout()
plt.show()

# =====
# SAVE B-H LOOP TO CSV FILE
# =====

np.savetxt(
    "BHloop.csv",
    BHloop,
    delimiter=",",
    header="H_A_per_m,B_T",
    comments=""
)
```

7.10 Примеры численных расчетов петель гистерезиса

Ниже мы приводим примеры расчетов петель гистерезиса нашей Python-имплементацией алгоритма.

Parameters.txt:

```
# Magnetic material parameters

Bs=1.6
Br=1.2
Hc=120.0
```

main.py

```
# =====
# GENERATE H(t)
# =====

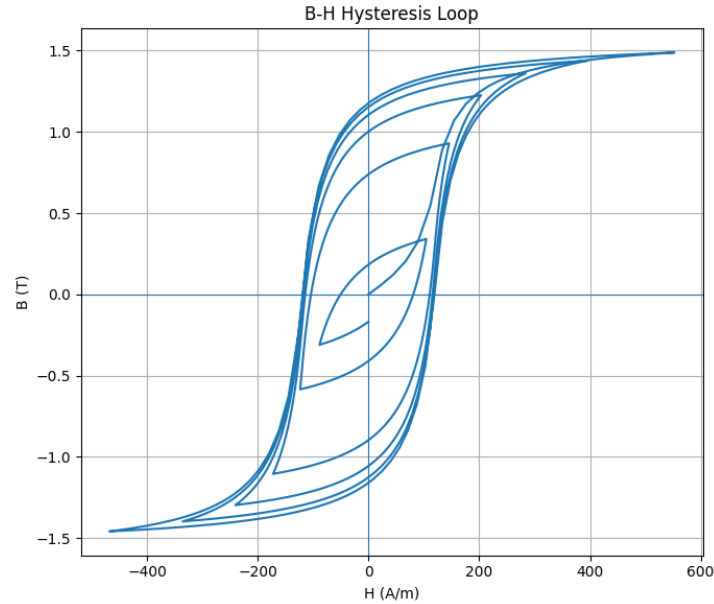
t_max = 0.1 # time, s
t = np.linspace(0, t_max, 1000)

# Choose a function Hscan(1,2,...) from Hscan.py
Hvalues = Hscan1(t)
```

Hscan.py

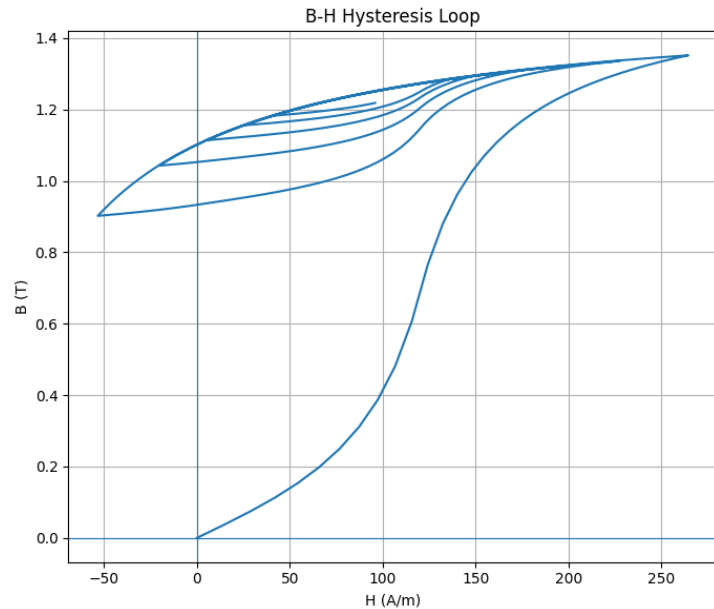
```
def Hscan1(t):
    w = 120 * pi
    T = (2 * pi) / w
    return 600 * np.sin(w * t) * np.exp(-t / (T * 3))

# Testing asymmetrical loops
Hc = par["Hc"]
```



Сканирующее поле: $Hscan1(t) = 600 \sin(120\pi t) \exp[-t/(3T)]$, $T = 2\pi/(120\pi) = 1/60$ s.

Синусоидальное поле 60 Hz с начальной амплитудой 600 A/m и экспоненциальной постоянной затухания $3T = 0.05$ s. Такой сигнал естественно создаёт последовательность уменьшающихся по амплитуде петель вокруг нуля и полезен для общей проверки memory behavior.



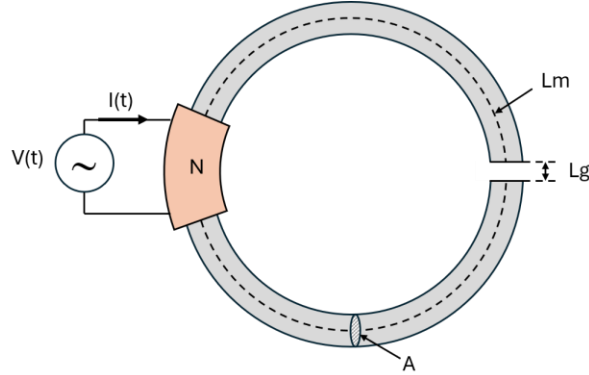
Сканирующее поле: $Hscan2(t) = Hbias(1 - \exp(-\tau/\tau_{bias})) + A0 \exp(-\tau/\tau_{decay}) \sin(\omega\tau)$

В текущем коде $Hbias = 0.8Hc$, $A0 = 1.5Hc$, $n_cycles = 5$, $\tau_{bias} = 0.01Tsim$, $\tau_{decay} = 0.8 * Tsim$, где $Tsim$ — время симуляций. Цель $Hscan2$ — создать несколько разворотов вокруг положительного смещения и тем самым чаще активировать affine-correction branches.

8. Python-проект для расчета переходных токов в индукторе с магнитным сердечником

Чтобы прояснить вычислительный алгоритм, используемый в LTspice при моделировании индукторов с ферромагнитными сердечниками, а также одновременно проверить разработанную реализацию модели магнитного гистерезиса на Python, ниже рассматриваются основные соотношения для таких индукторов и численный метод решения соответствующей системы уравнений. Основы теории магнитных цепей предполагаются известными, поэтому здесь приводятся только те положения, которые непосредственно необходимы для построения расчетного алгоритма. Python-проект сохранен в папке «Chan-Engelhardt-Model-Inductor» в репозитории [GitHub](#). [4]

Геометрия индуктора показана ниже. Форма сердечника и его однородное поперечное сечение могут быть произвольными. Мы выбрали кольцо. Также допускается наличие щели в сердечнике.



Запишем уравнения для напряженностей магнитного поля и тока в катушке $I(t)$, которые связаны между собой:

$$\begin{cases} H_m(t)L_m + H_g(t)L_g = NI(t) \\ RI(t) = V(t) - NA \frac{dB(t)}{dt} \end{cases} \quad (1)$$

где H_m – напряженность магнитного поля (A/m) внутри магнитного сердечника, H_g – напряженность магнитного в щели (A/m), R – сопротивление катушки (Ω), $V(t)$ – внешний источник эдс (V), и $-NA \frac{dB(t)}{dt}$ – эдс самоиндукции катушки. При одинаковом сечении сердечника магнитная индукция $B(t)$ будет равномерно распределенной вдоль него в силу условия непрерывности ее нормальной компоненты на границах раздела. Однако необходимо учесть ее зависимость от напряженности магнитного поля. В качестве независимого магнитного параметра выберем $H_m(t)$, тогда $B(t) = B(H_m(t))$. В пределах щели можно сразу записать $H_g(t) = B(H_m(t))/\mu_0$. В результате получаем следующую систему уравнений:

$$\begin{cases} I(t) = \frac{L_m}{N} H_m(t) + \frac{L_g}{\mu_0 N} B(H_m(t)) \\ RI(t) = V(t) - NA \frac{dB(H_m(t))}{dt} \end{cases} \quad (2)$$

Подставляя ток из первого уравнения во второе, получаем нелинейное дифференциальное уравнение относительно напряженности магнитного поля, замыкаемое гистерезисной зависимостью $B(H_m(t))$:

$$NA \frac{dB(H_m(t))}{dt} + \frac{RL_m}{N} H_m(t) + \frac{RL_g}{\mu_0 N} B(H_m(t)) = V(t) \quad (3)$$

Для численного решения (3) воспользуемся методом конечных разностей [Backward Euler](#) для временного шага $t_{n+1} = t_n + \Delta t$:

$$NA \frac{B(H_{m,n+1}) - B(H_{m,n})}{\Delta t} + \frac{RL_m}{N} H_{m,n+1} + \frac{RL_g}{\mu_0 N} B(H_{m,n+1}) = V_{n+1} \quad (4)$$

где $H_{m,n} = H_m(t_n)$, $H_{m,n+1} = H_m(t_n + \Delta t)$, и $V_{n+1} = V(t_n + \Delta t)$.

Для численного решения (4) вводится функция невязки:

$$F(H) = NA \frac{B(H) - B(H_{m,n})}{\Delta t} + \frac{RL_m}{N} H + \frac{RL_g}{\mu_0 N} B(H) - V_{n+1}. \quad (5)$$

Искомое значение магнитного поля определяется из условия $F(H_{m,n+1}) = 0$. После нахождения $H_{m,n+1}$ на очередном шаге вычисляются магнитная индукция и ток:

$$B_{n+1} = B(H_{m,n+1}) \quad (6)$$

$$I_{n+1} = \frac{L_m}{N} H_{m,n+1} + \frac{L_g}{\mu_0 N} B(H_{m,n+1}) \quad (7)$$

Особенность задачи состоит в том, что вычисление $B(H)$ связано с моделью гистерезиса, обладающей памятью. Состояние модели на момент t_n можно условно обозначить через S_n . Оно включает текущее положение на петле гистерезиса, направление предыдущего движения, сведения о разворотах, параметры минорных петель и другие внутренние величины, необходимые модели, которые мы обсуждали выше при разборе диаграмм алгоритма. Во время решения нелинейного уравнения по невязке $F(H)$ состояние S_n должно оставаться неизменным. Каждое пробное значение поля H_{trial} рассматривается как независимая гипотеза о состоянии системы в момент t_{n+1} . Для каждого такого значения выполняется полный алгоритм гистерезиса: $(H_{\text{trial}}, S_n) \rightarrow (B_{\text{trial}}, S_{\text{trial}})$. При этом модель может продолжить текущую ветвь гистерезиса либо определить, что между шагами n и $n+1$ произошёл разворот, и перейти на соответствующую новую ветвь. Все переходы выполняются точно так же, как при обычном последовательном расчёте гистерезисной траектории. Однако полученное состояние S_{trial} является только пробным. Оно не заменяет состояние S_n , пока соответствующее значение H_{trial} не удовлетворит нелинейному уравнению. Следующее пробное значение поля снова рассчитывается исходя из того же самого состояния S_n : $(H_{\text{trial},1}, S_n) \rightarrow (B_{\text{trial},1}, S_{\text{trial},1})$, $(H_{\text{trial},2}, S_n) \rightarrow (B_{\text{trial},2}, S_{\text{trial},2})$, и так далее. Нельзя использовать состояние, полученное на одной пробной итерации, в качестве исходного состояния для следующей пробной итерации. В противном случае последовательность точек, генерируемая методом поиска корня, сама стала бы искусственной магнитной предысторией и изменила бы петлю гистерезиса. Таким образом, при поиске корня выполняется полный алгоритм модели гистерезиса, но все возвращаемые ею новые параметры сохраняются как пробные. Только после нахождения окончательного значения $H_{m,n+1}$ соответствующее пробное состояние принимается как новое физическое состояние системы: $S_{n+1} = S_{\text{trial}}$.

Для применения [метода бисекции](#) при поиске корня нелинейного уравнения $F(H) = 0$ необходимо найти интервал $[H_a, H_b]$, на концах которого функция невязки имеет разные знаки: $F(H_a)F(H_b) \leq 0$. В качестве первой границы естественно принять известное значение поля на предыдущем временном шаге: $H_a = H_{m,n}$. При этом $B(H_a) = B(H_{m,n})$, поэтому производный член в функции невязки обращается в нуль. Следовательно, $F(H_{m,n}) = RI_n - V_{n+1}$. Величина $V_{n+1} - RI_n$ определяет физически ожидаемое направление изменения магнитной индукции. Действительно, из электрического уравнения $NA \frac{dB}{dt} = V - RI$ следует, что при $V_{n+1} - RI_n > 0$ магнитная индукция имеет тенденцию возрасти, а при $V_{n+1} - RI_n < 0$ — уменьшиться. Поэтому сначала поиск второй границы интервала проводится в соответствующем направлении от $H_{m,n}$. Задаётся начальный шаг поиска ΔH_0 . Первая пробная граница выбирается как $H_b = H_{m,n} + s\Delta H_0$, где:

$$s = \begin{cases} +1, & V_{n+1} - RI_n > 0, \\ -1, & V_{n+1} - RI_n < 0. \end{cases}$$

В точке H_b вычисляется значение $B(H_b)$ при сохранённом состоянии S_n , после чего определяется $F(H_b)$. Если выполняется условие $F(H_a)F(H_b) \leq 0$, то корень заключён в найденном интервале и можно переходить к методу бисекции для его поиска с предустановленной точностью и ограничением на максимальное число шагов. Если знаки невязки на концах одинаковы, расстояние от $H_{m,n}$ до второй границы увеличивается в два раза: $\Delta H_0, 2\Delta H_0, 4\Delta H_0, 8\Delta H_0, \dots$. Таким образом, $H_b^{(j)} = H_{m,n} + s 2^j \Delta H_0$. На каждом таком пробном значении снова выполняется независимый расчёт гистерезиса из одного и того же сохранённого состояния S_n . Если в физически ожидаемом направлении интервал со сменой знака найти не удалось, для повышения надёжности расчёта поиск повторяется в противоположном направлении. При этом используется тот же алгоритм последовательного удвоения расстояния от $H_{m,n}$. Такой дополнительный поиск особенно полезен около точек разворота петли гистерезиса, где простая оценка направления по величине $V_{n+1} - RI_n$ может оказаться недостаточной.

После нахождения интервала $[H_a, H_b]$, удовлетворяющего условию $F(H_a)F(H_b) \leq 0$, начинается собственно [метод бисекции](#). На каждой итерации вычисляется середина интервала: $H_c = \frac{H_a + H_b}{2}$. Для этого пробного значения, начиная из неизменного состояния S_n , выполняется полный расчёт модели гистерезиса и определяется $B_c = B(H_c)$, после чего вычисляется невязка $F_c = F(H_c)$. Если $F(H_a)F(H_c) \leq 0$, то корень находится в интервале $[H_a, H_c]$, и принимается $H_b = H_c$. Если же $F(H_a)F(H_c) > 0$, то корень находится в интервале $[H_c, H_b]$, и принимается $H_a = H_c$. После этого процедура повторяется. Поиск завершается, когда выполняется одно из условий сходимости: $|F(H_c)| \leq \varepsilon_F$, или $|H_b - H_a| \leq \varepsilon_H$, где ε_F — допустимая невязка нелинейного уравнения, а ε_H — допустимая ширина интервала поиска по напряжённости магнитного поля. Если за заданное максимальное число итераций необходимая точность не достигнута, временной шаг считается нерешённым. В этом случае следует уменьшить Δt , изменить параметры поиска интервала либо проверить параметры магнитной и электрической моделей. Не следует автоматически принимать последнее приближение, поскольку оно может не удовлетворять исходному уравнению с требуемой точностью.

После нахождения корня принимается $H_{m,n+1} = H_c$. Одновременно с этим принимаются соответствующие этому корню значения (6) и (7), а пробное состояние модели гистерезиса становится новым подтверждённым состоянием: $S_{n+1} = S_{\text{trial}}$. Только после этого временной шаг считается завершённым и состояние $H_{m,n+1}$, B_{n+1} , I_{n+1} , S_{n+1} используется как исходное при переходе к следующему моменту времени. В результате весь алгоритм можно представить в виде следующей последовательности:

$$\left[(H_{m,n}, B_n, I_n, S_n) \rightarrow V_{n+1} \rightarrow \text{поиск интервала} \rightarrow F(H_{m,n+1}) = 0 \rightarrow (H_{m,n+1}, B_{n+1}, I_{n+1}, S_{n+1}) \right].$$

Принципиальной особенностью метода является то, что все пробные расчёты внутри поиска корня выполняются при неизменном исходном состоянии гистерезиса S_n , а новое состояние фиксируется только после окончательного решения нелинейного уравнения.

8.1 Структура Python-проекта

Python-проект для расчёта переходных токов в индукторе является развитием проекта расчёта петель гистерезиса, рассмотренного в секции 7. Основное отличие состоит в том, что напряжённость магнитного поля H теперь не задаётся извне функцией $H_{\text{scan}}(t)$, а определяется на каждом временном шаге из уравнений электрической и магнитной цепей. При этом модуль `BH.py`, реализующий алгоритм `Chan–Engelhardt`, используется без изменения.

Структура проекта имеет вид:

```
project/
├─ main.py
├─ BH.py
├─ V.py
├─ Parameters.py
└─ Parameters.txt
```

`BH.py` отвечает за вычисление магнитной индукции и изменение состояния модели гистерезиса; `V.py` задаёт внешнее напряжение; `Parameters.py` загружает параметры из текстового файла `Parameters.txt`; `main.py` организует временной цикл, решение нелинейного уравнения и сохранение результатов.

8.2 Начальное состояние

Расчёт начинается из полностью размагниченного состояния. В `main.py` одновременно задаются начальные значения электрических и магнитных переменных и состояние модели гистерезиса:

```
Hlast = 0.0
Blast = 0.0
Ilast = 0.0

OnInitMag = True
IsLatched = False
```

```
Bd = 0.0
Bd_turn = 0.0

H0 = 0.0
scale = 0.0
offset = 0.0
```

```
previous_direction = None
```

Таким образом, первая точка соответствует началу координат на initial magnetization curve. Переменные Hlast, Blast и llast далее всегда относятся к последнему уже подтверждённому временному шагу.

Временная сетка строится с постоянным шагом dt, а программа дополнительно проверяет, что полное время моделирования является целым числом временных шагов:

```
n_steps = int(round(par["simulation_time"] / par["dt"]))

if not np.isclose(
    n_steps * par["dt"],
    par["simulation_time"]
):
    raise ValueError(
        "simulation_time must be an integer multiple of dt"
    )

t = np.arange(n_steps + 1, dtype=float) * par["dt"]
```

8.3 Пробное вычисление магнитного состояния

При решении нелинейного уравнения на очередном временном шаге приходится многократно проверять различные пробные значения поля Htrial. Для этого введена функция evaluate_hysteresis_trial(). Сначала определяется направление изменения поля относительно последнего подтверждённого значения:

```
if Htrial > Hlast:
    ...
    result = BH_increasing(...)
или
result = BH_decreasing(...)
```

Если направление движения изменилось относительно предыдущего временного шага, функции BH_increasing() или BH_decreasing() самостоятельно выполняют все необходимые переходы алгоритма Chan–Engelhardt, включая формирование новой минорной ветви и, при необходимости, affine correction.

Принципиально важно, что результат вызова сохраняется во временных переменных:

```
Btrial = result[0]
trial_state = result[1:]
```

и не записывается немедленно в основное состояние модели.

Иными словами, каждое пробное значение Htrial рассматривается как возможное следующее состояние системы, но до окончания поиска корня оно остаётся только гипотезой.

8.4 Сохранение состояния гистерезиса при поиске корня

Вся процедура поиска корня выполняется при неизменных значениях Hlast, Blast, OnInitMag, IsLatched, Bd, Bd_turn, H0, scale, offset, previous_direction, соответствующих последнему подтверждённому временному шагу. Функции модели гистерезиса могут для конкретного Htrial вернуть изменённые параметры состояния, однако они сохраняются только в trial_state. Следующая проба вновь начинается из того же исходного состояния. Это позволяет избежать принципиальной ошибки: точки, которые рассматриваются методом поиска корня, не должны сами образовывать фиктивную историю изменения магнитного поля.

8.5 Функция невязки

Для каждого Htrial сначала вычисляется соответствующее значение магнитной индукции, затем пробный ток и значение невязки:

```
def residual(Htrial, Vnext):

    Btrial, trial_state, trial_direction = \
        evaluate_hysteresis_trial(Htrial)

    Itrial = (
        par["I_Fe"] * Htrial
        + (par["g"] / mu0) * Btrial
    ) / par["N"]

    F = (
        par["R"] * Itrial
        + par["N"] * par["A"]
        * (Btrial - Blast) / par["dt"]
        - Vnext
    )

    return F, Btrial, Itrial, trial_state, trial_direction
```

Таким образом, функция residual() возвращает не только значение невязки, необходимое численному solver'у, но и всё пробное состояние системы, соответствующее данному значению Htrial.

8.6 Поиск интервала для бисекции

В начале нового временного шага невязка вычисляется в уже известной точке Hlast:

```
F0, B0, I0, state0, direction0 = residual(Hlast, Vnext)
```

Затем определяется наиболее вероятное направление поиска корня:

```
drive = Vnext - par["R"] * Ilast

if drive > 0.0:
    primary_direction = 1.0
elif drive < 0.0:
    primary_direction = -1.0
```

Сначала поиск производится именно в этом направлении. От H_{last} делается пробный шаг $initial_H_step$, после чего шаг последовательно удваивается:

```
Hb = Hlast + search_direction * step
Fb, _, _, _ = residual(Hb, Vnext)

if Fa * Fb <= 0.0:
    bracket_found = True
    break

step *= 2.0
```

Как только невязка меняет знак, найден интервал, содержащий корень.

Для повышения надёжности алгоритма, если интервал не найден в первоначально предполагаемом направлении, поиск полностью повторяется в противоположную сторону:

```
for search_direction in (
    primary_direction,
    -primary_direction
):
```

Это особенно полезно около точек разворота гистерезисной траектории.

8.7 Метод бисекции

После нахождения интервала начинается стандартная процедура бисекции. На каждой итерации выбирается его середина: $H_m = 0.5 * (H_a + H_b)$. Для неё заново вычисляется полное пробное магнитное состояние:

```
(
    Fm,
    Bm,
    Im,
    state_m,
    direction_m
) = residual(Hm, Vnext)
```

После этого сохраняется та половина интервала, в которой происходит смена знака невязки:

```
if Fa * Fm <= 0.0:
    Hb = Hm
    Fb = Fm
else:
    Ha = Hm
    Fa = Fm
```

Расчёт заканчивается при достижении заданной точности по невязке либо по ширине интервала:

```
if (
    abs(Fm) <= par["residual_tol"]
    or abs(Hb - Ha) <= par["H_tol"]
):
    return best
```

Если максимальное количество итераций исчерпано без достижения требуемой точности, программа генерирует ошибку, а последнее приближение не принимается автоматически как физическое состояние.

8.8 Фиксация нового состояния

Только после успешного решения нелинейного уравнения пробное состояние становится реальным состоянием системы на новом временном шаге:

```
(
    OnInitMag,
    IsLatched,
    Bd,
    Bd_turn,
    H0,
    scale,
    offset
) = state_next
```

Затем сохраняются рассчитанные ток, поле и магнитная индукция:

```
Ivalues[n + 1] = Inext
Hvalues[n + 1] = Hnext
Bvalues[n + 1] = Bnext

Hlast = Hnext
Blast = Bnext
Ilast = Inext
```

Только после этого начинается следующий временной шаг. Этим обеспечивается фундаментальное разделение между пробным состоянием, используемым внутри нелинейного solver'a, и подтверждённым состоянием, представляющим реальную магнитную историю системы.

8.9 Источник напряжения и параметры

В отличие от программы секции 7, где задавалась тестовая функция $H(t)$, в модели индуктора внешним воздействием служит напряжение. Его закон вынесен в самостоятельный модуль V.py:

```
def V(t):
    return (
        par["V_offset"]
        + par["V_amplitude"]
        * np.sin(2.0 * np.pi * par["V_frequency"] * t)
    )
```

Это позволяет впоследствии менять форму возбуждающего напряжения без каких-либо изменений в численном алгоритме расчёта магнитной цепи.

Все численные параметры, как и в предыдущем проекте, вынесены в Parameters.txt и загружаются через Parameters.py:

```
parameters = {}

with open(file_path, "r", encoding="utf-8") as f:
    for line in f:
        ...
        name, value = line.split("=", 1)
        parameters[name.strip()] = float(value.strip())
```

Таким образом, непосредственно в вычислительных модулях отсутствуют численные параметры конкретного эксперимента.

8.10 Результаты расчёта

В процессе интегрирования сохраняются три временные последовательности: ток катушки, напряжённость магнитного поля и магнитная индукция. После завершения моделирования ток записывается в файл Current.csv:

```
np.savetxt(
    "Current.csv",
    np.column_stack((t, Ivalues)),
    delimiter=",",
    header="time_s,I_A",
    comments=""
)
```

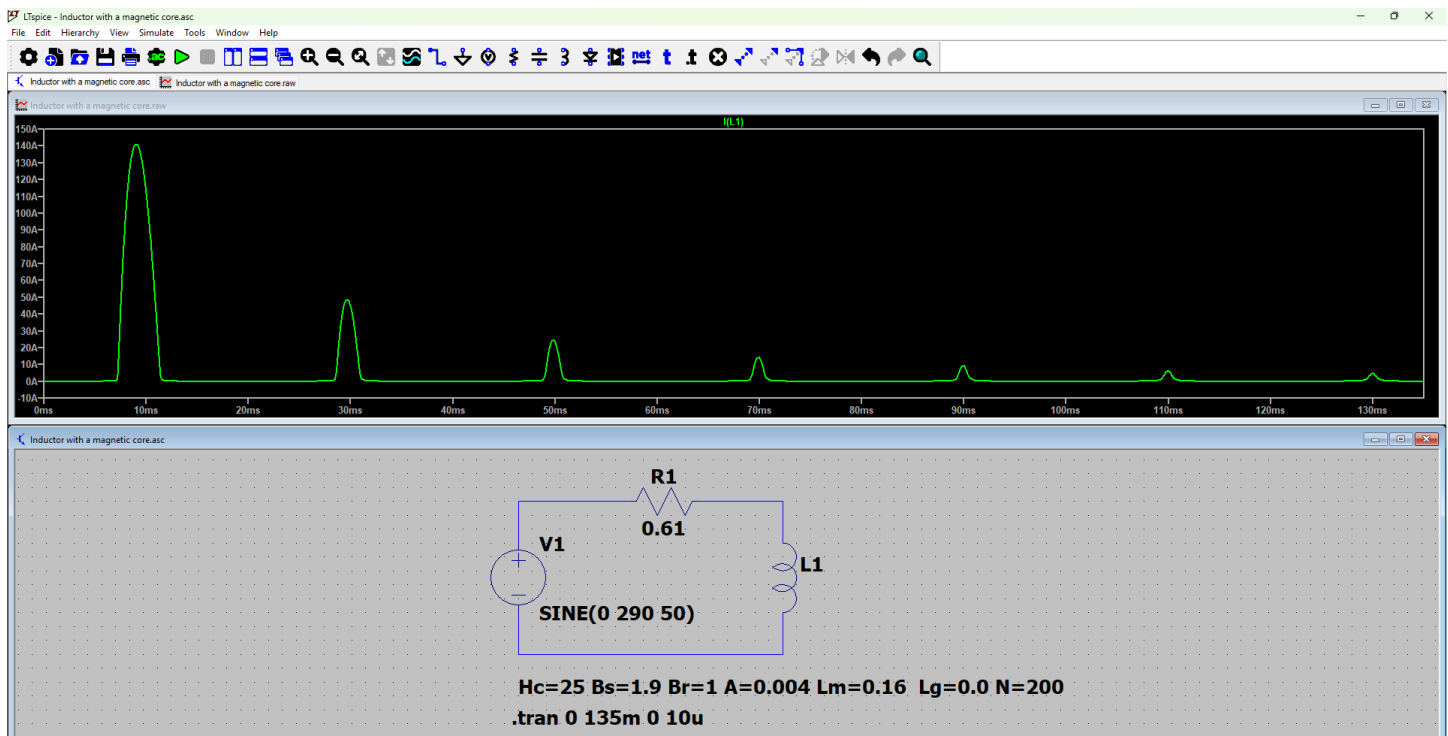
Траектория гистерезиса сохраняется отдельно в BHloop.csv:

```
np.savetxt(
    "BHloop.csv",
    np.column_stack((Hvalues, Bvalues)),
    delimiter=",",
    header="H_A_per_m,B_T",
    comments=""
)
```

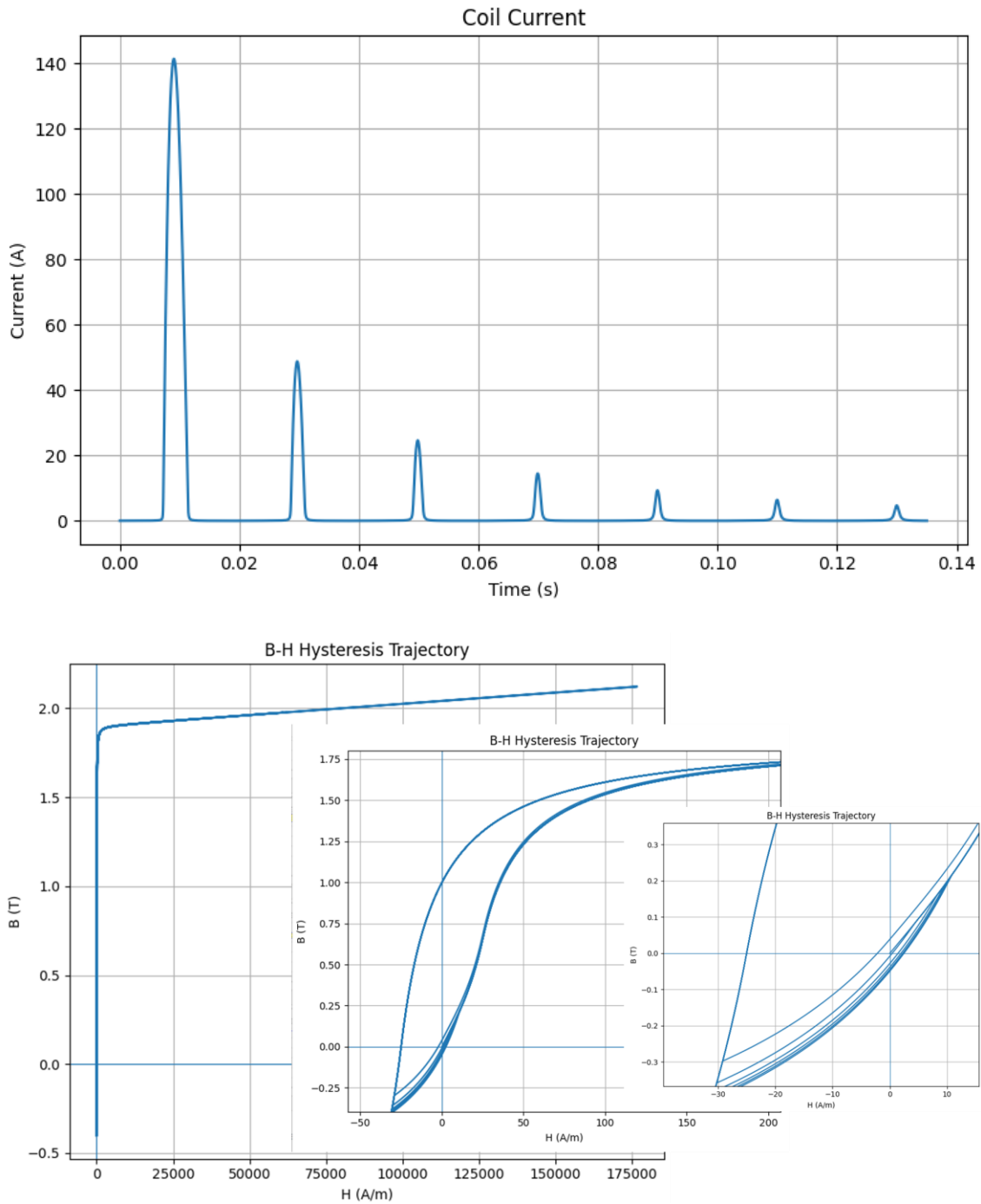
Программа также строит графики $I(t)$ и полной траектории $B(H)$. Последний особенно полезен при отладке: он позволяет одновременно с электрическим откликом контролировать, какие мажорные, минорные и асимметричные ветви выбирает модель гистерезиса.

8.11 Примеры расчетов переходного тока в индукторе и сравнение с LTspice

Приведенная ниже схема и переходный ток в индукторе с магнитным сердечником без щели приведены в патенте. Мы воспроизвели ее в LTspice (папка «LTspice_Inductor with a magnetic core» внутри папки «Chan-Engelhardt-Model-Inductor»).



Наш расчет, показанный ниже, полностью совпадает с LTspice.



```
# =====
# MAGNETIC MATERIAL PARAMETERS
# =====

# Saturation magnetic flux density, Bs [T]
Bs=1.9

# Remanent magnetic flux density, Br [T]
Br=1.0

# Coercive magnetic field strength, Hc [A/m]
Hc=25.0
```

```

# =====
# MAGNETIC CORE AND COIL PARAMETERS
# =====

# Number of coil turns, N [-]
N=200

# Coil resistance, R [ohm]
R=0.61

# Mean magnetic path length in the ferromagnetic core, l_Fe [m]
l_Fe=0.16

# Effective magnetic cross-sectional area, A [m^2]
A=0.004

# Air-gap length, g [m]
g=0.0

# =====
# VOLTAGE SOURCE PARAMETERS
# =====

# DC voltage offset [V]
V_offset=0.0

# Sinusoidal voltage amplitude [V]
V_amplitude=290.0

# Voltage frequency [Hz]
V_frequency=50.0

# =====
# SIMULATION PARAMETERS
# =====

# Total simulation time measured from t = 0 [s]
simulation_time=0.135

# Time step [s]
dt=0.000001

# =====
# NONLINEAR SOLVER PARAMETERS
# =====

# Tolerance for the electrical-equation residual F(H) [V]
residual_tol=1.0e-9

# Tolerance for magnetic field strength H [A/m]
H_tol=1.0e-10

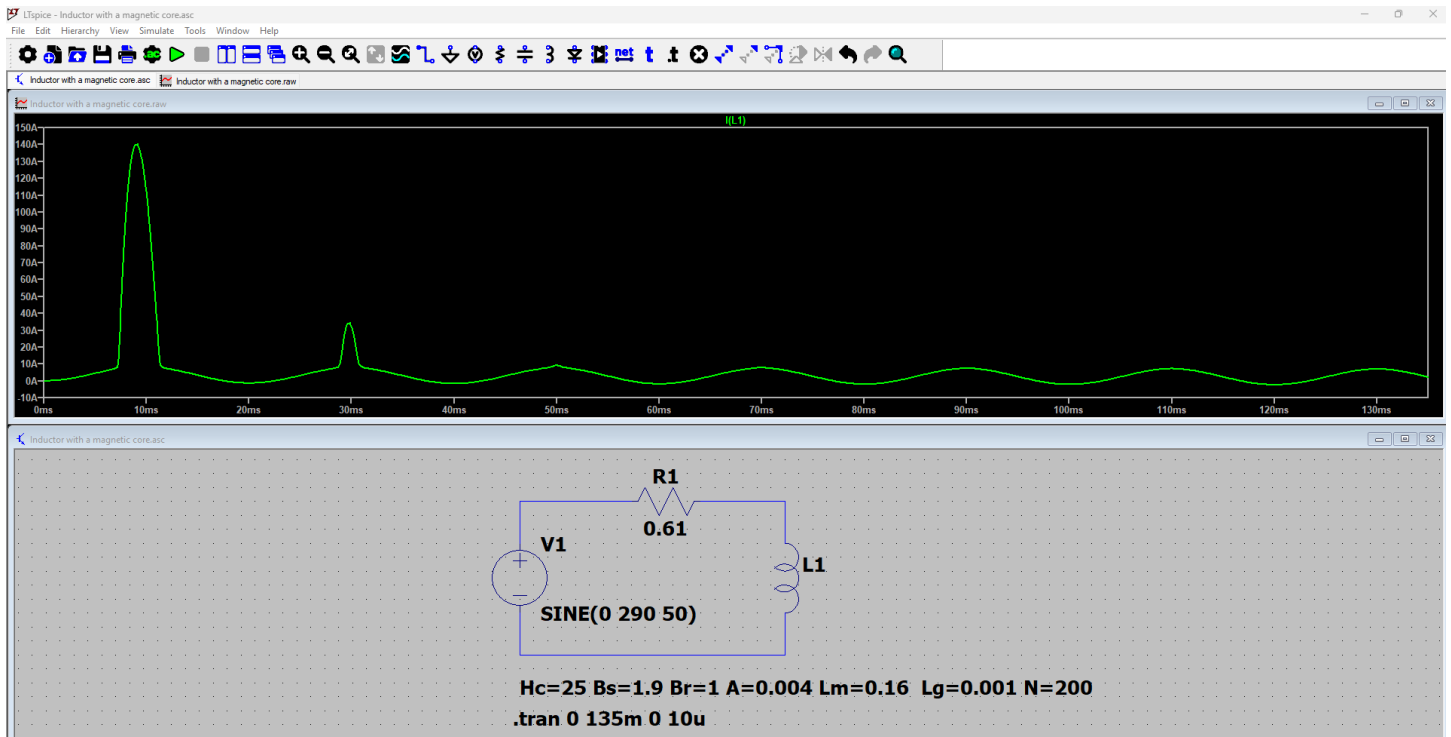
# Maximum number of root-bracketing iterations [-]
max_bracket_iter=80

# Maximum number of bisection iterations [-]
max_bisection_iter=120

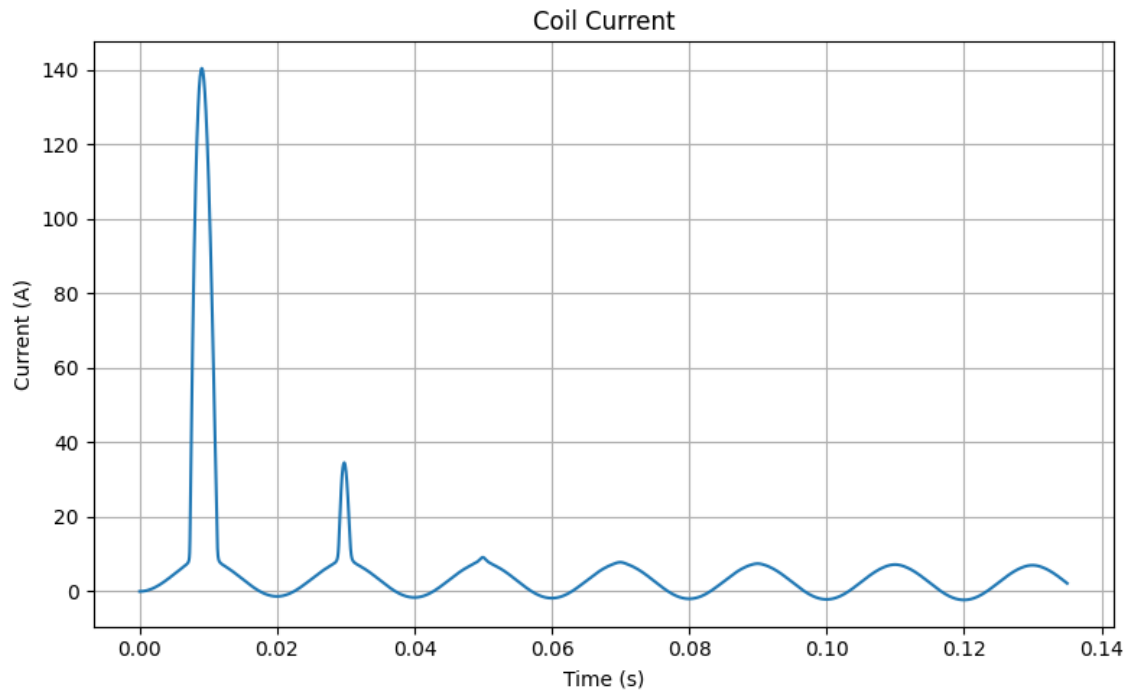
# Initial magnetic-field increment used to search for a root bracket [A/m]
initial_H_step=6.0

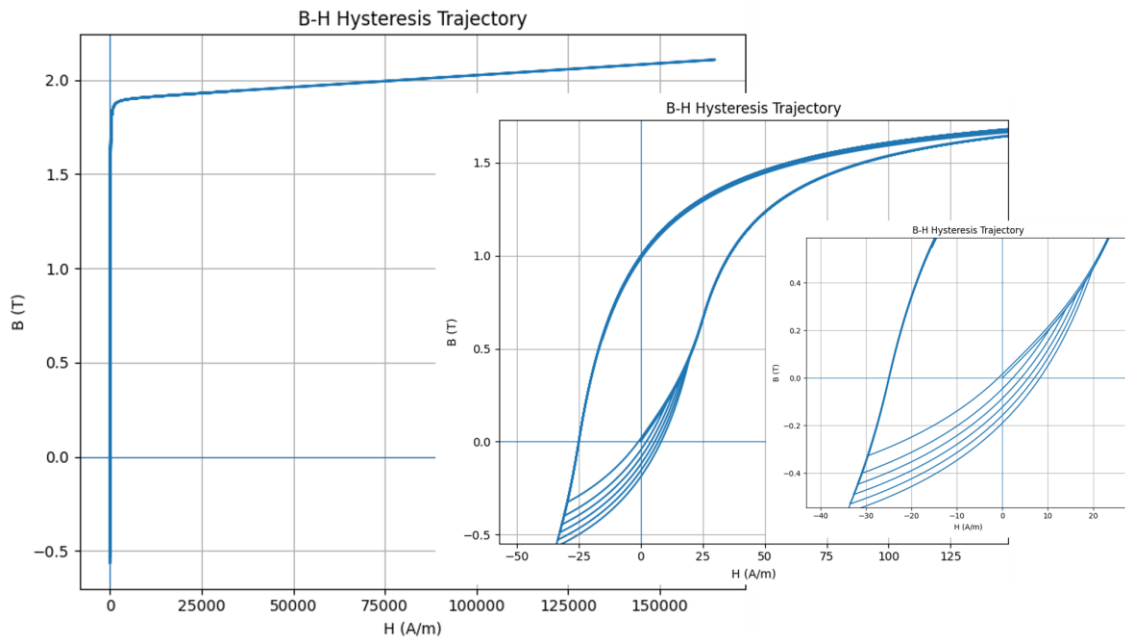
```

Теперь проведем расчет для того же магнитного индуктора, но с щелью 1 мм.



Наш расчет, показанный ниже, полностью совпадает с LTspice.





Источники

- [1] M. Th. Engelhardt, «*Asymmetric Minor Hysteresis Loop Model and Circuit Simulator Including the Same*», U.S. Patent No. 7502723B1, Mar. 10, 2009.
- [2] John H. Chan et al., «*Nonlinear Transformer Model for Circuit Simulation*», IEEE Transactions on Computer-Aided Design, vol. 10, No. 4, APRIL 1991.
- [3] D. C. Jiles and D. L. Atherton, « Theory of ferromagnetic hysteresis», Journal of Magnetism and Magnetic Materials 61 (1986) 48-60.
- [4] Python-проекты на GitHub: <https://github.com/DmitriyMakhnovskiy/Chan-Engelhardt-Model-BHloops>